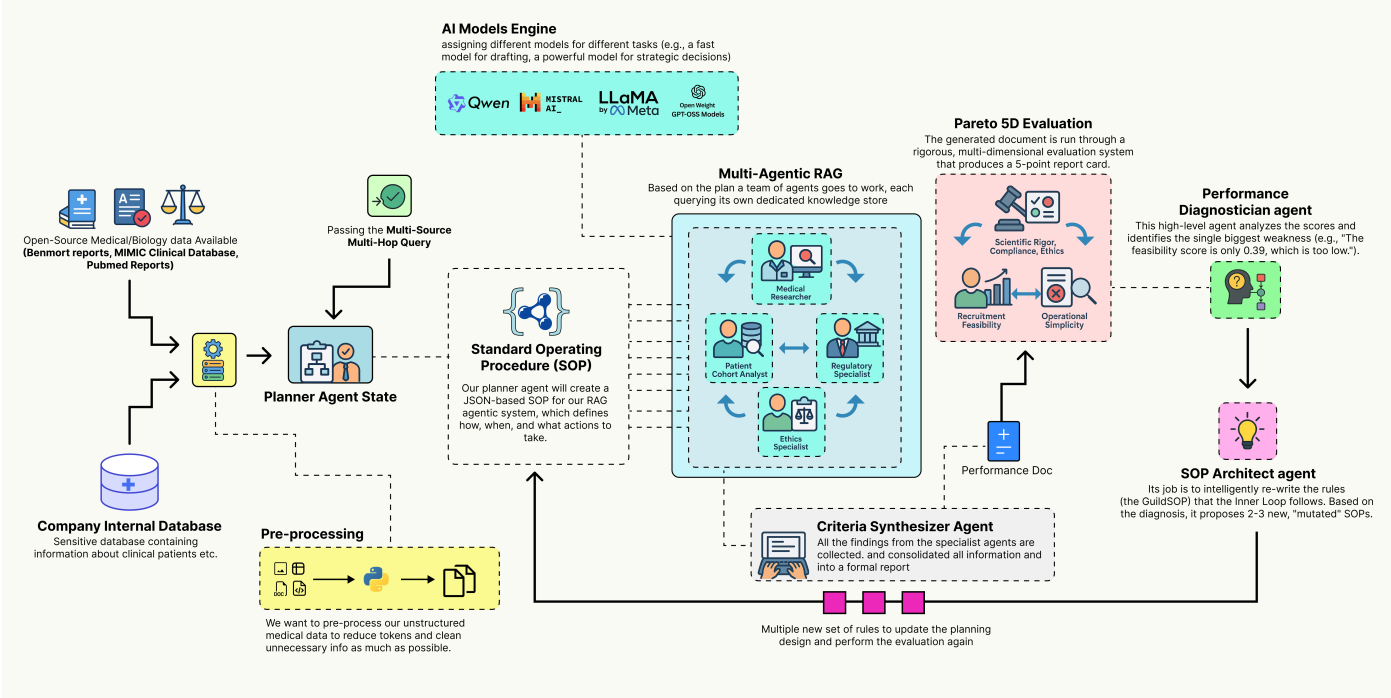
# Building a Self-Improving Agentic RAG System

levelup.gitconnected.com/building-a-self-improving-agentic-rag-system-f55003af44c4

Fareed Khan November 14, 2025




Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

You're reading for free via Friend Link. Become a member to access the best of Medium.

Member-only story

## Specialist agents, multi-dimensional eval, Pareto front and more.

Read this story for free: link

Agentic RAG systems act as a **high dimensional vector space** where each dimension represents a design decision such as prompt engineering, agent coordination, retrieval strategies, and much more. Manually tuning these dimensions to find the right combination is extremely difficult and unseen data in production often breaks what worked in testing.

**A better approach is to let the system learn how to optimize itself**. A typical Agentic RAG pipeline that **evolves itself** follows the thinking process as shown below:

- A collaborative team of carries out the task. It takes a high-level concept and generates a complete, multi-source document using its current standard operating procedures.

- A scores the team output, measuring performance across multiple goals such as accuracy, feasibility, and compliance, producing a performance vector.
- A performance analyzes this vector, acting like a consultant to identify the main weakness in the process and trace its root cause.
- An uses this insight to update the procedures, proposing new variations specifically designed to fix the identified weakness.
- Each is tested as the team repeats the task, with each output evaluated again to produce its own performance vector.
- The system identifies the , the best trade-offs among all tested SOPs and presents these optimized strategies to a , completing the evolutionary loop.

In this blog, we are going to target the **healthcare domain**, which is very challenging because **multiple possibilities** need to be considered based on the input query or the knowledge base, **while the final decision remains in the hands of a human.**

> We will build a complete end-to-end, self-improving Agentic RAG pipeline that generates different design patterns for RAG systems.

All the code is available in my GitHub repository:

## GitHub - FareedKhan-dev/autonomous-agentic-rag: Self improving agentic rag pipeline

### Self improving agentic rag pipeline. Contribute to FareedKhan-dev/autonomous-agentic-rag development by creating an…

github.com

## Table of Contents

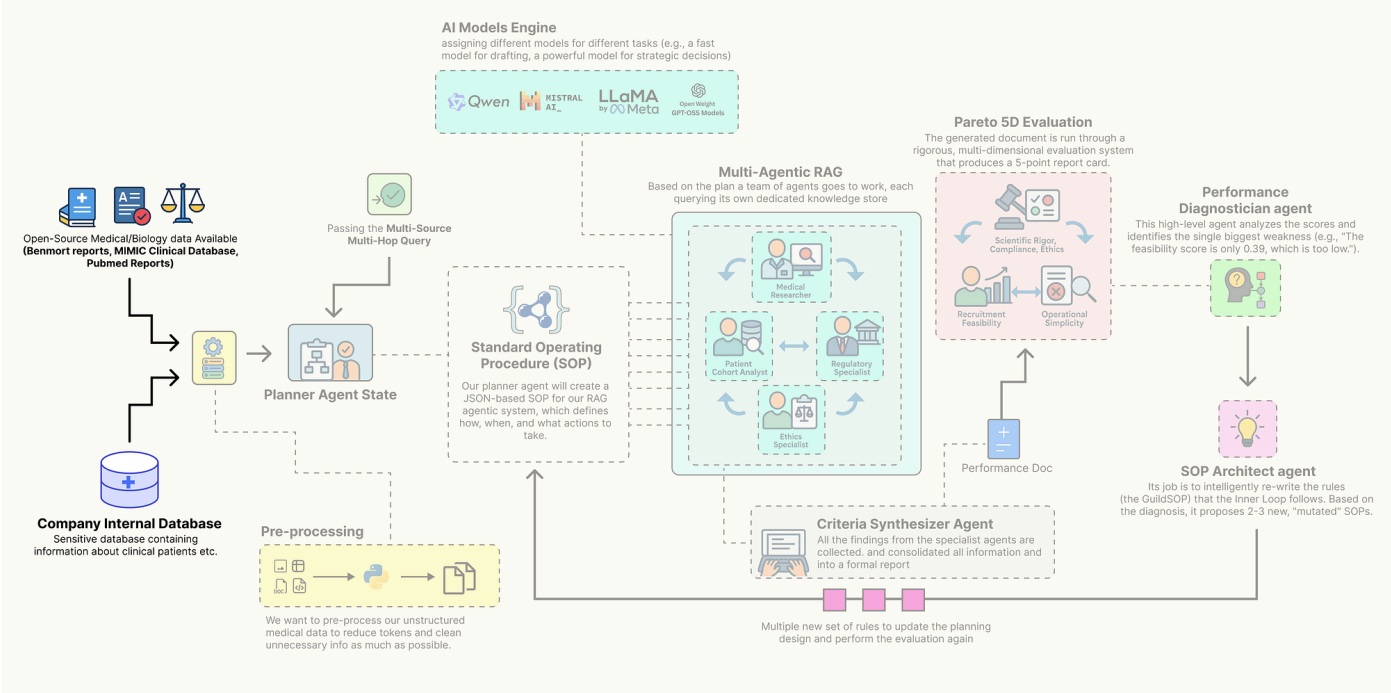


## Knowledge Infrastructure for Medical AI

Before we can code our self-evolving agentic RAG system, we need to establish a proper knowledge database along with the necessary tools required to build the architecture.

A production-grade RAG system typically contains a diverse set of databases, including sensitive organizational data as well as open-source data, to improve retrieval quality and compensate for outdated or incomplete information. This foundational step is arguably the most critical …

> as the quality of our data sources will directly determine the quality of our final output.




In this section, we are going to assemble every component of this architecture. Here is what we are going to do:

- We will set up our environment with all the necessary libraries, focusing on a local, open-source-first approach.
- Then going to securely load our API keys and configure `LangSmith` to trace and debug our complex agent interactions from the very beginning.
- We are going to build a suite of different open-source models using `Ollama`, assigning specific models to specific tasks to optimize for performance and cost.
- downloading and preparing four real-world data sources: scientific literature from PubMed, regulatory guidelines from the FDA, ethical principles, and a massive structured clinical dataset (MIMIC-III).
- Finally, we will process this raw data into highly efficient, searchable databases, `FAISS` vector stores for our unstructured text and a `DuckDB` instance for our structured clinical data.

## Installing the Open-Source Stack

So, our first step is to install all the required Python libraries. A reproducible environment is the bedrock of any serious project. We are selecting a industry-standard, open-source stack that gives us full control over our system. This includes `langchain` and `langgraph` for the core agentic

framework, `ollama` for interacting with our local LLMs, and specialized libraries like `biopython` for accessing PubMed and `duckdb` for high-performance analytics on our clinical data.

Let's install the required modules …

```
%pip install -U langchain langgraph langchain_community langchain_openai langchain_core
ollama pandas duckdb faiss-cpu sentence-transformers biopython pypdf pydantic lxml
html2text beautifulsoup4 matplotlib -qqq
```

We are gathering all the tools and building materials we will need for the rest of the project in one go. Each library has a specific role, from agent workflows with `langgraph` to data analysis with `duckdb`.

Now that w have installed the required modules, let's start initializing them one by one.

## Environment Configuration & Imports

We need to securely configure our environment. Hardcoding API keys directly into a notebook is a significant security risk and makes the code difficult to share.

We will use a `.env` file to manage our secrets, primarily our `LangSmith` API key. Setting up `LangSmith` from the very beginning is non-negotiable for a project of this complexity, it provides the deep observability we will need to trace, debug, and understand the interactions between our agents. So, let's do that.

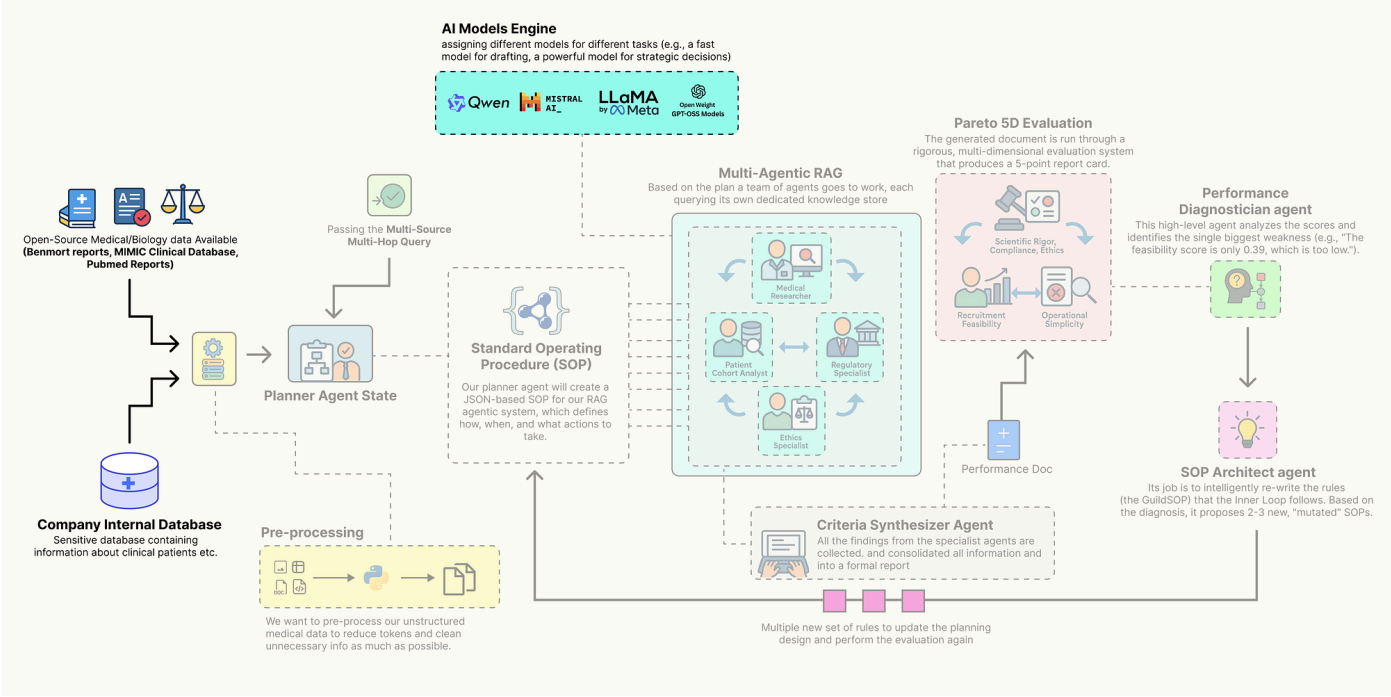
```
 os getpass dotenv  load_dotenvload_dotenv()    os.environ     os.environ:        ():
()os.environ[] =
```

The function `load_dotenv()` acts as a secure bridge between our sensitive credentials and our code. It reads the `.env` file (which should never be committed to version control) and injects the keys into our session environment.

> From this point forward, every operation we perform with LangChain or LangGraph will be automatically captured and sent to our project in LangSmith.

## Configuring the Local LLM

In production-grade agentic systems, a one-size-fits-all model strategy is rarely optimal. A massive, state-of-the-art model is computationally expensive and slow, using it for every simple task would be waste of resources especially if it's hosted on your GPUs. But a small, fast model might lack the deep reasoning power needed for high-stakes strategic decisions.




The key is to fit the right model at right place of your agentic system. We will build a group of different open-source models, each chosen for its strengths in a specific role, and all served locally via Ollama for privacy, control, and cost-effectiveness.

We need to define a configuration dictionary to hold the clients for each of our chosen models. This way we can easily swap models and centralizes our model management.

```
 langchain_community.chat_models  ChatOllama langchain_community.embeddings
OllamaEmbeddingsllm_config = {            : ChatOllama(model=, temperature=, =),
: ChatOllama(model=, temperature=),    : ChatOllama(model=, temperature=),
: ChatOllama(model=, temperature=, =),        : OllamaEmbeddings(model=)}
```

So we have just created our `llm_config` dictionary, which serves as a centralized hub for all our model initializations. By assigning different models to different roles, we are creating a cost-performance optimized hierarchy.

- The `planner`, `drafter`, and `sql_coder` roles handle frequent, well-defined tasks. Using smaller models like `Qwen2 7B` and `Llama 3.1 8B` for these roles ensures low latency and efficient resource usage. They are perfectly capable of following instructions to generate plans, draft text, or write SQL.
- The `director` agent has the most complex job, it must analyze multi-dimensional performance data and rewrite the entire system operating procedure. This requires deep reasoning and a understanding of cause and effect. For this high-stakes, low-frequency task, we allocate our most powerful resource, the `Llama 3 70B` model.

Let’s execute this cell to initialize the clients and print their configurations.

```
()()()()()()
```

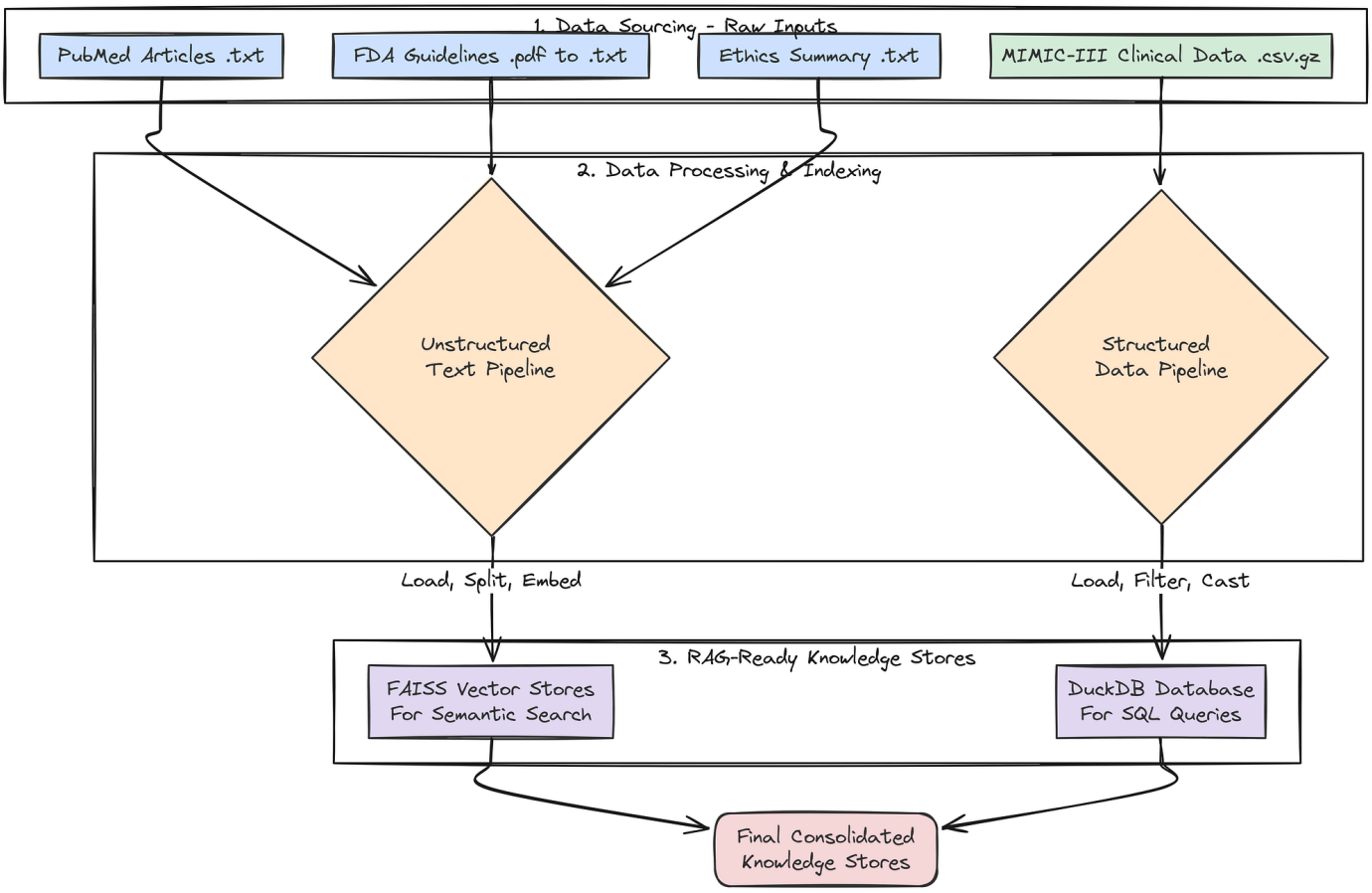
This is what we are getting …

```
LLM clients configured:Planner (llama3.1:8b-instruct): ChatOllama(model=, temperature=0.0, format=)Drafter (qwen2:7b): ChatOllama(model=, temperature=0.2)SQL Coder (qwen2:7b): ChatOllama(model=, temperature=0.0)Director (llama3:70b): ChatOllama(model=, temperature=0.0, format=)Embedding Model (nomic-embed-text): OllamaEmbeddings(model=)
```

The output confirms that our `ChatOllama` and `OllamaEmbeddings` clients have been successfully initialized with their respective models and parameters. now we are ready to be connected with our knowledge stores.

## Preparing the Knowledge Stores

RAG most important part is this, a rich multi-modal knowledge base to draw upon. A generic, web-based search is not enough for a specialized task like clinical trial design. We need to ground our agents in authoritative, domain-specific information.




To achieve this, we will now build a comprehensive **knowledge base** by sourcing, downloading, and processing four distinct types of real-world data. This multi-source approach is critical for enabling our agents to synthesize information and produce a comprehensive, well-rounded output.

First, a small but important step: we will create the directories where our downloaded and processed data will live.

```
 osdata_paths = {    : ,    : ,    : ,    : ,    : } path  data_paths.values():
os.path.exists(path):        os.makedirs(path)        ()
```

We are making sure our project has a clean and organized file structure from the start. By pre-defining and creating these directories, our subsequent data processing functions become more robust, they can reliably save their outputs to the correct location without needing to check if the directory exists first.

Next, we will fetch real scientific literature from PubMed. This will provide the core knowledge for our `Medical Researcher` agent, grounding its work in up-to-date, peer-reviewed science.

```
 Bio  Entrez Bio  Medline ():            Entrez.email = os.environ.get()    ()
handle = Entrez.esearch(db=, term=query, retmax=max_articles, sort=)    record =
Entrez.read(handle)    id_list = record[]    ()        ()        handle =
Entrez.efetch(db=, =id_list, rettype=, retmode=)    records = Medline.parse(handle)
count =          i, record  (records):        pmid = record.get(, )        title =
record.get(, )        abstract = record.get(, )         pmid:
filepath = os.path.join(data_paths[], )             (filepath, )  f:
f.write()            ()            count +=      count
```

The `download_pubmed_articles` function is our direct connection to the live scientific literature. It's a three-step process:

1. `esearch`to find relevant article IDs, `efetch` to download the full records.
2. Then a loop to parse and save the crucial information (Title and Abstract) into clean text files.

Let’s run this function with a query specific to our use case.

```
pubmed_query = num_downloaded = download_pubmed_articles(pubmed_query)()
```

When we run the above code, it will start downloading the pubmed articles highly relevant to our query.

```
Fetching PubMed articles  query: (SGLT2 inhibitor) AND ( 2 diabetes) AND (renal
impairment)Found 20 article IDs.Downloading articles...[1/20] Fetching PMID: 38810260...
Saved to ./data/pubmed_articles/38810260.txt[2/20] Fetching PMID: 38788484... Saved to
./data/pubmed_articles/38788484.txt...PubMed download complete. 20 articles saved.
```

It successfully connected to the NCBI database, executed our specific query, and downloaded 20 relevant scientific abstracts, saving each one into our designated `pubmed_articles` directory.

Our `Medical Researcher` agent will now has a rich, current, and domain-specific knowledge base to draw from, ensuring its findings are grounded in real science.

Now, let’s get the regulatory documents that our `Regulatory Specialist` agent will need. A key part of trial design is ensuring compliance with government guidelines.

```
 requests pypdf  PdfReader io ():        ()    :                response =
requests.get(url)        response.raise_for_status()
(output_path, )  f:            f.write(response.content)        ()
reader = PdfReader(io.BytesIO(response.content))        text =                  page
reader.pages:            text += page.extract_text() +
txt_output_path = os.path.splitext(output_path)[] +          (txt_output_path, )  f:
f.write(text)              requests.exceptions.RequestException  e:        ()
```

This function, `download_and_extract_text_from_pdf`, is our tool for handling PDF documents. It's a two-stage process.

1. First, it downloads and saves the original PDF from the FDA website. Second, and more importantly, it immediately processes that PDF using `pypdf` to extract all the text content.
2. It then saves this raw text to a `.txt` file. This pre-processing step is crucial because it converts the complex PDF format into simple text that our document loaders can easily ingest when we build our vector stores later on.

Let’s run the function to download our FDA guidance document.

```
fda_url = fda_pdf_path = os.path.join(data_paths[],
)download_and_extract_text_from_pdf(fda_url, fda_pdf_path)Downloading FDA Guideline:
https://www.fda.gov/media//downloadSuccessfully downloaded  saved to
./data/fda_guidelines/fda_diabetes_guidance.pdf
```

We now have both the original `fda_diabetes_guidance.pdf` and its extracted text version in our `fda_guidelines` directory. Our `Regulatory Specialist` agent is now equipped with its foundational legal and regulatory text.

Next, we will create a curated document for our `Ethics Specialist`. While we could search for this information, providing a concise, authoritative summary of core principles ensures the agent's reasoning is grounded in the most important concepts.

```
ethics_content = ethics_path = os.path.join(data_paths[], ) (ethics_path, )  f:
f.write(ethics_content)()
```

We have created a focused document for our `Ethics Specialist`. Instead of having the agent sift through the entire Belmont Report, we have provided it with the most critical information in a clean, easily digestible format. This ensures its analysis will be consistent and grounded in the core principles.

Now for our most complex data source: the structured clinical data from MIMIC-III. This will provide the real-world population data our `Patient Cohort Analyst` needs to assess recruitment feasibility.

```
 duckdb pandas  pd os ():        ()    db_path = os.path.join(data_paths[], )    csv_dir =
os.path.join(data_paths[], )            required_files = {        : os.path.join(csv_dir,
),        : os.path.join(csv_dir, ),        : os.path.join(csv_dir, ),    }
missing_files = [path  path  required_files.values()   os.path.exists(path)]
missing_files:        ()         f  missing_files: ()        ()                ()
os.path.exists(db_path):        os.remove(db_path)        con = duckdb.connect(db_path)
()    con.execute()        ()    con.execute()           ()               con.execute()
con.execute()        con.execute()    con.close()     db_path
```

Instead of trying to load the massive MIMIC-III CSV files into memory with pandas (which would likely crash), we are using`DuckDB` ability to process data directly from disk. The two-stage processing of the`LABEVENTS` table is a critical technique. By first loading the data as text and filtering it before casting to numeric types, before this we handle data quality issues and create a final table that is smaller, cleaner, and much faster to query.

Let's execute the function to build our clinical database and then run a quick test to inspect the result.

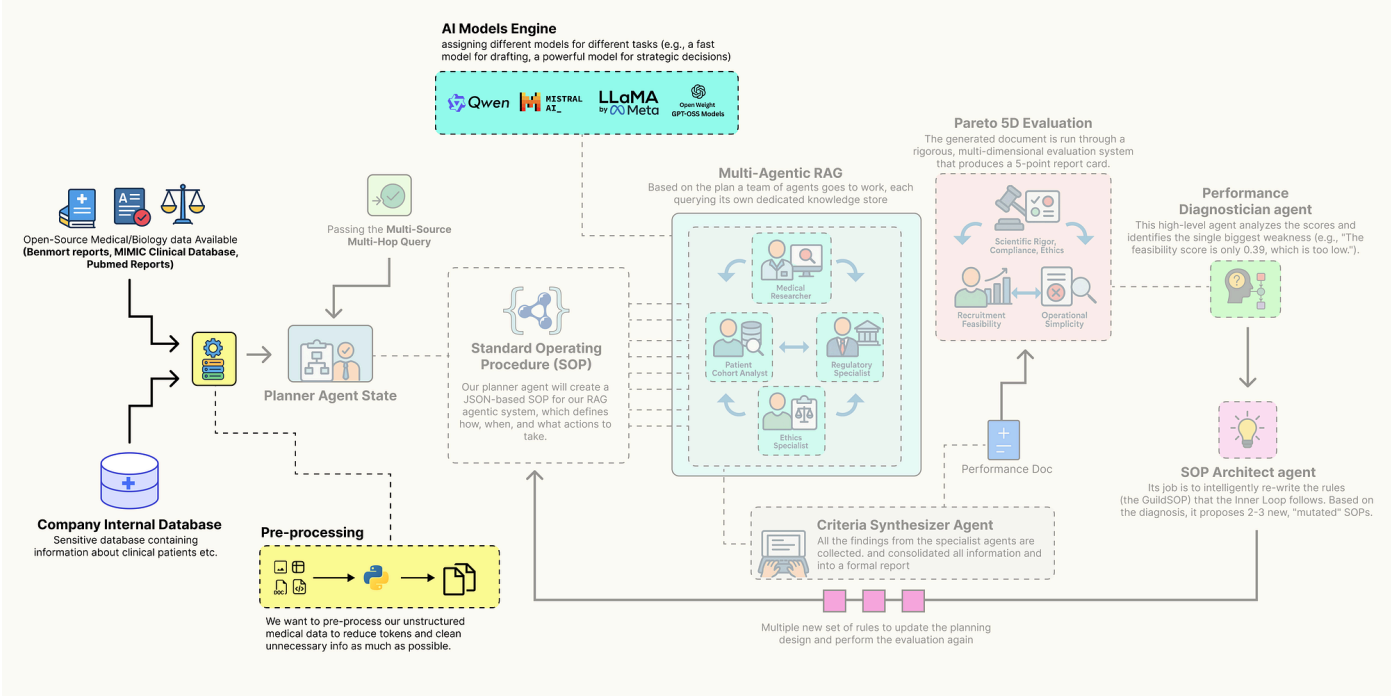
```
db_path = load_real_mimic_data() db_path:    ()    ()    con = duckdb.connect(db_path)
()    ()    (con.execute().df())    ()    (con.execute().df())    con.close()
```

The output we are getting …

```
Attempting to load real MIMIC-III data from  CSVs...Required files found. Proceeding with
database creation.Loading PATIENTS.csv.gz into DuckDB...Loading DIAGNOSES_ICD.csv.gz into
DuckDB...Loading and processing LABEVENTS.csv.gz (this may take several minutes)...Real
MIMIC-III database created at: ./data/mimic_db/mimic3_real.dbTesting database connection
and schema...Tables  DB: [, , ]Sample of  table:   ROW_ID  SUBJECT_ID GENDER         DOB
DOD    DOD_HOSP    DOD_SSN EXPIRE_FLAG0      238       250      F  2164-12-27  2198-02-18
2198-02-18 2198-02-18           11      239       251      M  2078-02-21         NaN
NaN        NaN           02      240       252      M  2049-06-06  2123-09-01  2123-09-01
2123-09-01           13      241       253      F  2081-11-26         NaN         NaN
NaN           04      242       254      F  2028-04-12         NaN         NaN        NaN
0Sample of  table:   ROW_ID  SUBJECT_ID  HADM_ID  SEQ_NUM ICD9_CODE0  129769       109
172335        1      403011  129770       109    172335        2      4862  129771
109    172335        3      582813  129772       109    172335        4      58554  129773
109    172335        5      42822
```

The output confirms that our data ingestion pipeline worked. We have successfully created a persistent `DuckDB` SQL database at `./data/mimic_db/mimic3_real.db`. The test queries show that the core tables (`patients`, `diagnoses_icd`, `labevents`) have been loaded correctly with the right schemas.

Our `Patient Cohort Analyst` agent now has access to a high-performance, real-world clinical database containing millions of records, enabling it to provide truly data-grounded feasibility estimates.




Finally, let's index all our unstructured text data into searchable vector stores. This will make the PubMed, FDA, and ethics documents accessible to our RAG agents.

```
 langchain_community.document_loaders  DirectoryLoader, TextLoader langchain.text_splitter
RecursiveCharacterTextSplitter langchain_community.vectorstores  FAISS
langchain_core.documents  Document ():        ()        loader =
DirectoryLoader(folder_path, glob=, loader_cls=TextLoader, show_progress=)    documents =
loader.load()          documents:        ()                        text_splitter =
RecursiveCharacterTextSplitter(chunk_size=, chunk_overlap=)    texts =
text_splitter.split_documents(documents)        ()    ()            db =
FAISS.from_documents(texts, embedding_model)    ()     db ():            pubmed_db =
create_vector_store(data_paths[], embedding_model, )    fda_db =
create_vector_store(data_paths[], embedding_model, )    ethics_db =
create_vector_store(data_paths[], embedding_model, )                      {        :
pubmed_db.as_retriever(search_kwargs={: })  pubmed_db  ,        :
fda_db.as_retriever(search_kwargs={: })  fda_db  ,        :
ethics_db.as_retriever(search_kwargs={: })  ethics_db  ,        : db_path     }
```

This `create_vector_store` function is the approach for creating RAG-ready knowledge bases from text files. It encapsulates the common **"load -> split -> embed -> index"** pattern. The `create_retrievers` function then orchestrates this process, creating a separate, specialized vector store for each of our document types.

Instead of a single, massive vector store, we have smaller, domain-specific stores. This allows our agents to perform more targeted and efficient searches (e.g., the `Regulatory Specialist` will only ever query the `fda_retriever`).

Let's run the final function to build our complete set of knowledge stores.

```
knowledge_stores = create_retrievers(llm_config[])() name, store
knowledge_stores.items():    ()

--- Creating PubMed Vector Store ---100%|██████████| 20/20 [00:00<00:00,
1102.77it/s]Loaded 20 documents,  into 35 chunks.Generating embeddings and indexing into
FAISS... (This may take a moment)Batches: 100%|██████████| 2/2 [00:03<00:00,
1.70s/it]PubMed Vector Store created successfully.--- Creating FDA Vector Store ---100%|
██████████| 1/1 [00:00<00:00, 137.95it/s]Loaded 1 documents,  into 48 chunks.Generating
embeddings and indexing into FAISS... (This may take a moment)Batches: 100%|██████████|
2/2 [00:04<00:00,  2.08s/it]FDA Vector Store created successfully.--- Creating Ethics
Vector Store ---100%|██████████| 1/1 [00:00<00:00, 143.20it/s]Loaded 1 documents,  into 1
chunks.Generating embeddings and indexing into FAISS... (This may take a moment)Batches:
100%|██████████| 1/1 [00:00<00:00,  2.62it/s]Ethics Vector Store created
successfully.Knowledge stores and retrievers created successfully.pubmed_retriever:
VectorStoreRetriever(tags=[, ], vectorstore=<...>)fda_retriever:
VectorStoreRetriever(tags=[, ], vectorstore=<...>)ethics_retriever:
VectorStoreRetriever(tags=[, ], vectorstore=<...>)mimic_db_path:
./data/mimic_db/mimic3_real.db
```

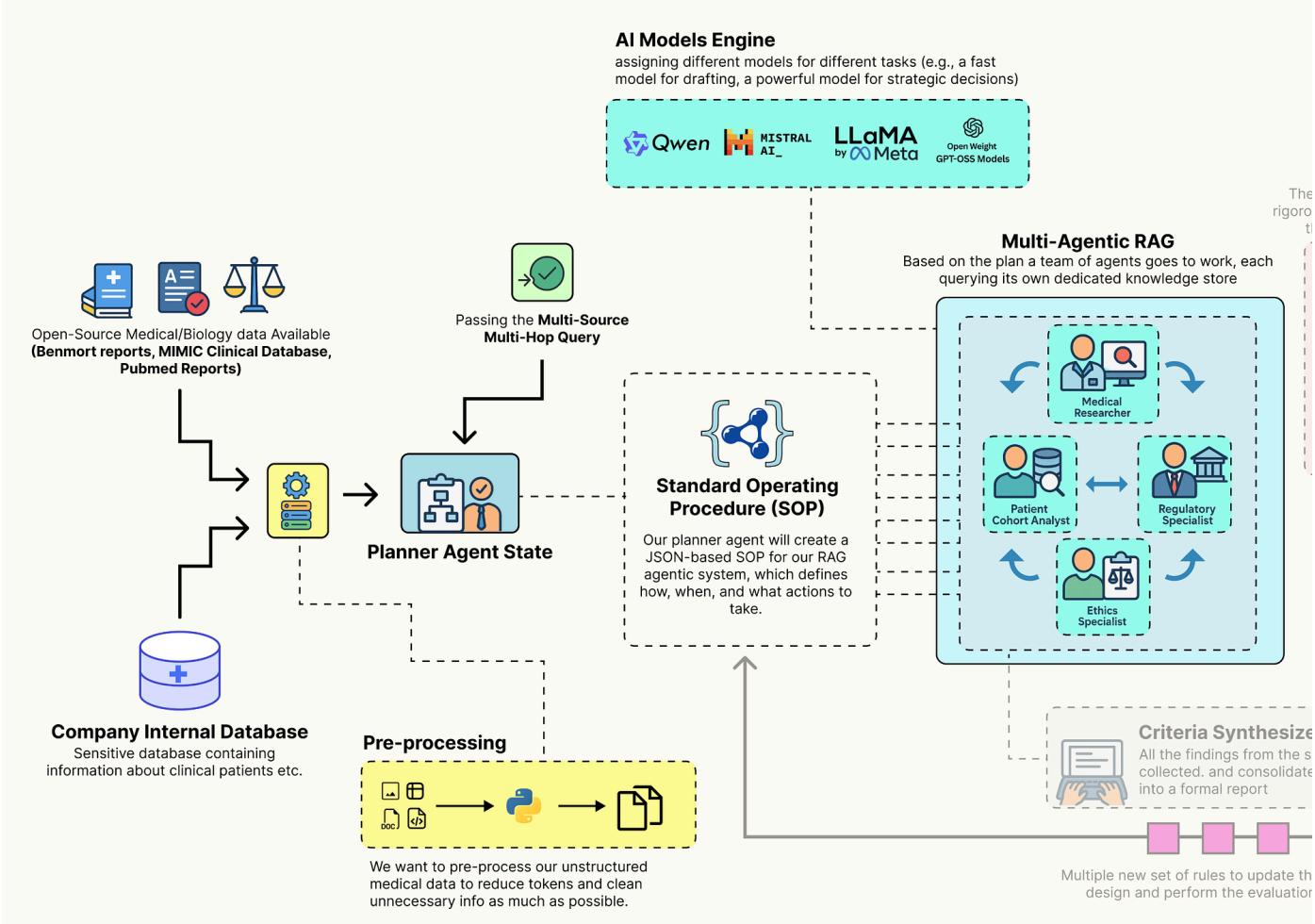
The output confirms that our entire knowledge is now fully assembled and operational. We have successfully processed all our unstructured text sources, PubMed, FDA, and Ethics into searchable `FAISS` vector stores.

The final `knowledge_stores` dictionary is our complete, centralized repository of data access tools. It contains everything our agent guild will need to perform its research.

With our data downloaded, processed, and indexed, and our LLMs configured, we can now begin constructing the first major component of our agentic system: The Trial Design Guild.

## Building The Inner Trial Design Network

With our knowledge base is now ready, we can now construct the core of our system. This is not going to be a simple, linear RAG chain. It is a collaborative, multi-agent workflow built with `LangGraph`, where a team of AI specialists works together to transform a high-level trial concept into a detailed, data-grounded criteria document.




The behavior of this entire architecture is not hardcoded. Instead, it is governed by a single, dynamic configuration object we call the **Standard Operating Procedure (`GuildSOP`)**.

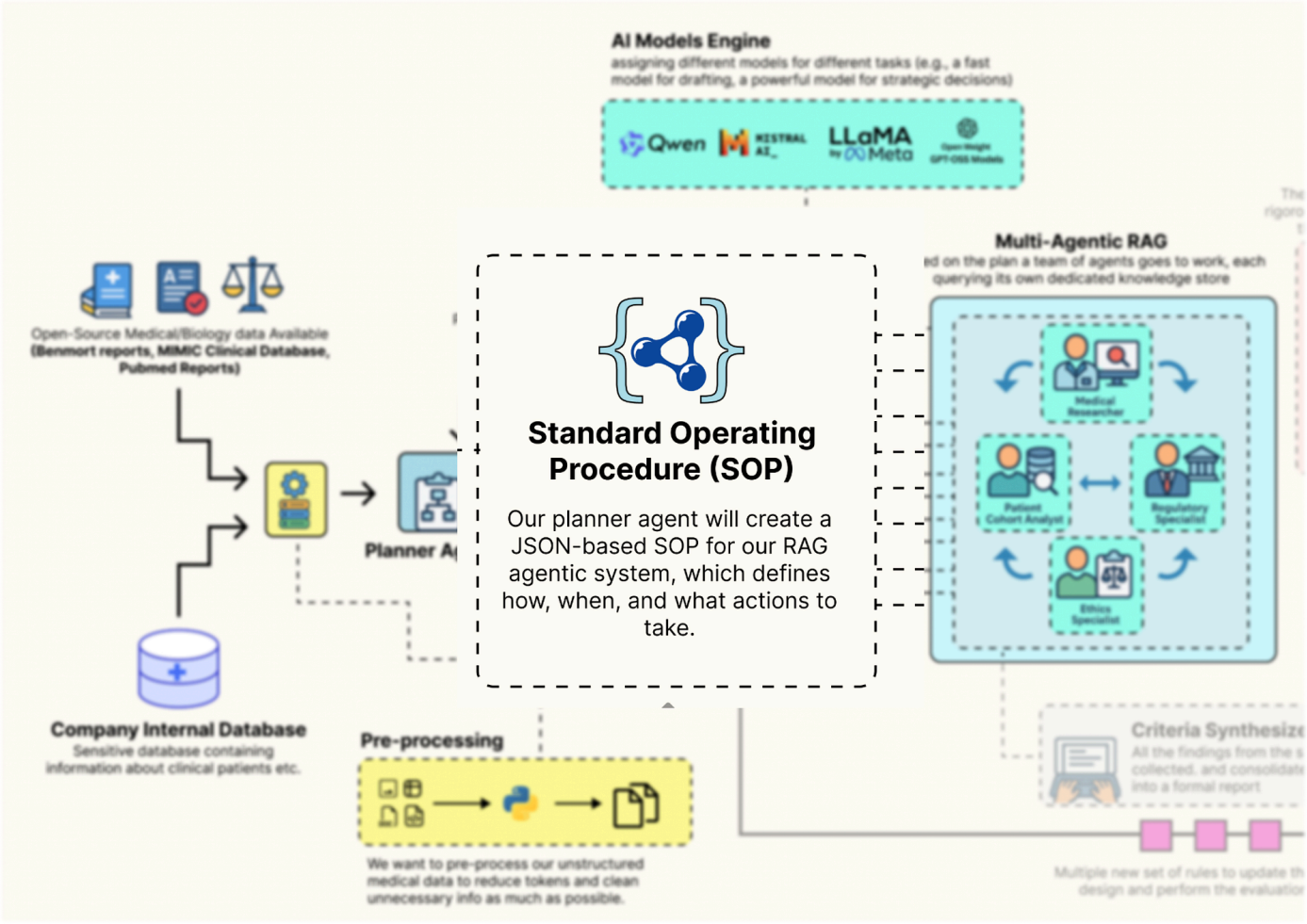
This SOP is the **"genome"** of our RAG pipeline, and it is this genome that our outer-loop **"AI Research Director"** will learn to evolve and optimize.

In this section, here is what we are going to do:

- We will create the `GuildSOP` Pydantic model, a structured configuration that will control every aspect of the tag architecture workflow.
- We will define the `GuildState`, the central space where our agents will share their plans and findings.
- We will implement each specialist, the Planner, the Researchers, the SQL Analyst, and the Synthesizer as a distinct Python function that will serve as a node in our graph.
- We will wire these agent nodes together using `LangGraph` to define the complete, end-to-end workflow of the Guild.
- We are also going to invoke the entire compiled Guild graph with our baseline SOP to see it in action and generate our first criteria document.

## Defining the Guild SOP

First, we need to define the structure that will control the entire behavior flow. We will use a Pydantic `BaseModel` to create our `GuildSOP`. This is a crucial design choice. Using Pydantic gives us a typed, validated, and self-documenting configuration object.




This `GuildSOP` is the central part that our outer-loop AI Director will later mutate and evolve, so having a strict schema is important for a stable evolutionary process. Let's code that.

```
 pydantic  BaseModel, Field typing   ():                planner_prompt:  =
Field(description=)           researcher_retriever_k:  = Field(description=, default=)
synthesizer_prompt:  = Field(description=)           synthesizer_model: [, ] =
Field(description=, default=)           use_sql_analyst:  = Field(description=, default=)
use_ethics_specialist:  = Field(description=, default=)
```

The `GuildSOP` class is more than just a configuration file, it's a live document that defines the Guild current strategy. By exposing key parameters like prompts, retriever settings (`researcher_retriever_k`), and even which agents to use (`use_sql_analyst`), we are creating a set of strategies that our outer-loop AI Director can pull to tune the entire performance.

We are using `Literal` for `synthesizer_model` to make sure the type safety so that the Director can only choose from a pre-defined list of valid models.

Now that we have the blueprint for our SOP, let's create a concrete, version 1.0 instance. This `baseline_sop` will be our starting point, the initial, hand-engineered strategy that we will task our AI Director with improving.

```
 jsonbaseline_sop = GuildSOP(        planner_prompt=,            synthesizer_prompt=,
researcher_retriever_k=,           synthesizer_model=,           use_sql_analyst=,
use_ethics_specialist=)
```

The prompts we have written are highly specific for getting reliable, structured behavior from LLMs. This baseline represents our best initial guess at an effective strategy. The performance of this SOP will serve as the benchmark that our AI Director will try to beat.

Let's run this to create our baseline object and print it out for inspection.

```
()(json.dumps(baseline_sop.(), indent=))
```

This is what we get when we run the above code …

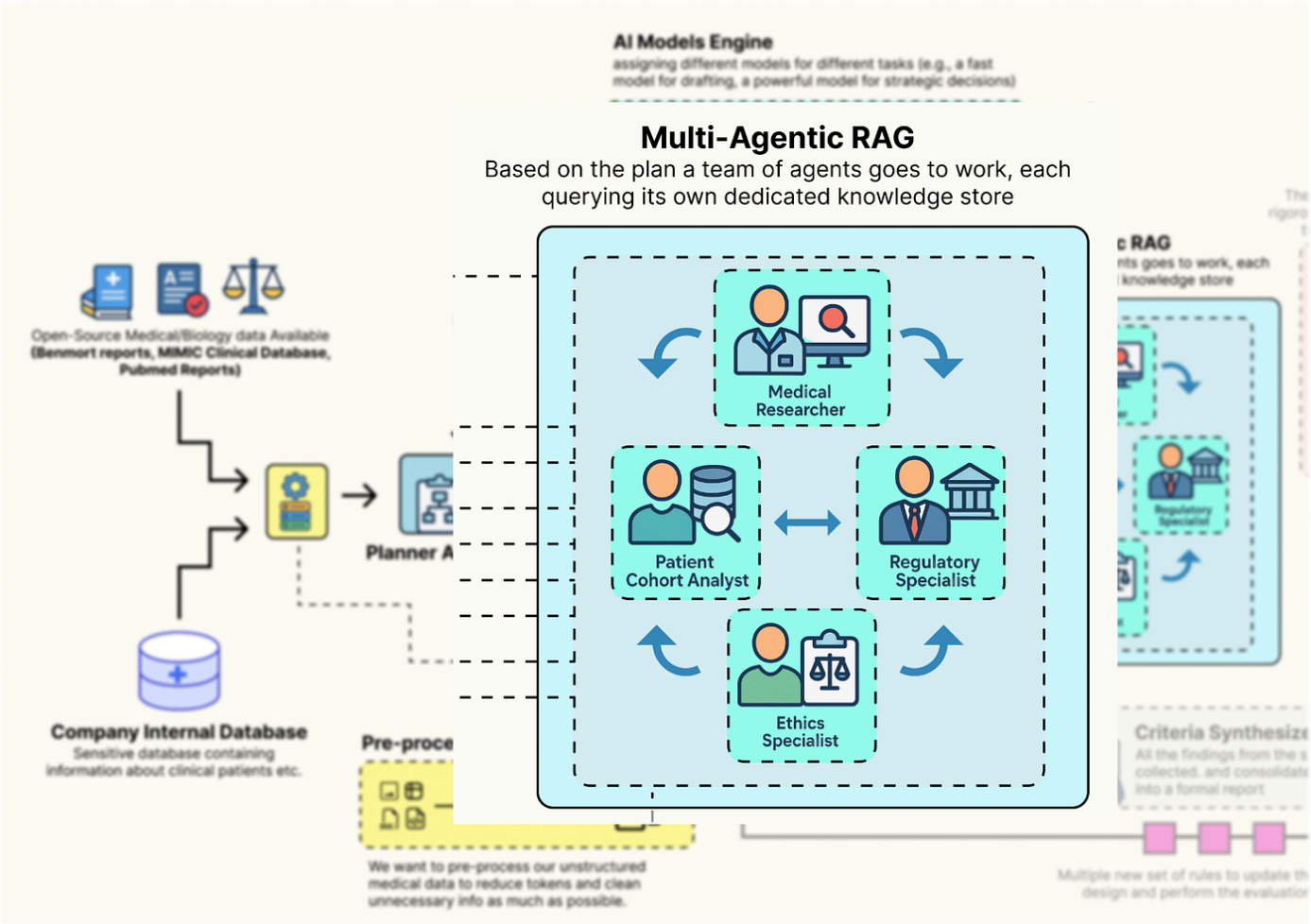
```
Baseline GuildSOP (v1.0):{    : ,    : 3,    : ,    : ,    : ,    : }
```

The output shows our fully instantiated baseline SOP as a clean JSON object. We can see all the configuration parameters that will now guide our Guild first run.

For example, the `planner_prompt` clearly outlines the expected output, and we can see that the `researcher_retriever_k` is set to `3`. If our system later struggles with insufficient context, our AI Director could learn to increase this value. This object is the "source code" for our agentic process, and we've just created our first version.

## Defining the Specialist Agents

Now that we have the **rulebook** (the SOP), we need to define the agents themselves. In `LangGraph`, agents are represented as nodes, which are simply Python functions that take the current graph state as input and return an update to that state.




First, we must define the structure of that state. This `GuildState` will be the shared **workbench** or **whiteboard** that all our agents use to collaborate. It will hold the initial request, the planner generated plan, the collected findings from each specialist, and the final output.

```
 typing  , , ,  langchain_core.pydantic_v1  BaseModel typing_extensions  TypedDict ():
agent_name:     findings:  ():        initial_request:                      plan: [[, ]]
agent_outputs: [AgentOutput]          final_criteria: []             sop: GuildSOP
```

The `AgentOutput` class is making sure that as specialists complete their work, their findings are neatly packaged and labeled. The `GuildState TypedDict` is the master blueprint for our shared memory. It's the "workbench" where the `plan` is laid out, the `agent_outputs` are collected like puzzle pieces, and the `final_criteria` is ultimately assembled.

Crucially, the `sop` is part of the state itself. This means we can inject a different SOP for every run of the graph, allowing our outer loop to test different strategies by simply changing this one object in the initial input.
Now, let's build our first agent: the Planner. This agent is the entry point for the Guild. It takes the user's high-level request and, guided by the `planner_prompt` in the SOP, creates a structured, step-by-step plan for the other specialists.

```
 () -> GuildState:        ()        sop = state[]        planner_llm = ll-
config[].with_structured_output(schema={: []})           prompt =     ()
response = planner_llm.invoke(prompt)    ()            {**state, : response}
```

It reads its own instructions (`planner_prompt`) from the `sop` object passed in the state. It then uses the `.with_structured_output()` method to force the LLM to return a valid JSON plan. This is a highly robust pattern that avoids the flakiness of manually parsing natural language outputs. The function concludes by returning the updated state, now containing the `plan` that will guide the subsequent agents.

We now need to build the specialist agents that will execute its plan. To avoid writing repetitive code, we'll start by creating a generic, reusable function for all our RAG-based specialists (the Medical Researcher, Regulatory Specialist, and Ethics Specialist).

```
 () -> AgentOutput:        ()    ()           retriever =
knowledge_stores[retriever_name]                agent_name == :
retriever.search_kwargs[] = state[].researcher_retriever_k        ()        retrieved_docs
= retriever.invoke(task_description)           findings = .join([  doc  retrieved_docs])
()    ()            AgentOutput(agent_name=agent_name, findings=findings)
```

The `retrieval_agent` function is a reusable component for creating RAG specialists. Instead of writing separate functions for each researcher, we have created a single, configurable agent. It takes the `retriever_name` as an argument and dynamically selects the correct knowledge base (PubMed, FDA, etc.) to query. The most important feature is how it interacts with the `GuildSOP`.

It specifically checks if it's acting as the **Medical Researcher** and, if so, adjusts its retrieval parameter `k` based on the value in `state['sop'].researcher_retriever_k`. This makes the thoroughness of the literature search a dynamically tunable parameter that our AI Director can evolve.

Now, let's build our most technically complex specialist: the Patient Cohort Analyst. This agent will bridge the gap between unstructured RAG and structured data analytics. It will take a natural language request, use an LLM to translate it into a valid SQL query, and then execute that query against our DuckDB database of MIMIC-III data to provide a data-grounded feasibility estimate.

```
 langchain_core.prompts  ChatPromptTemplate langchain_core.output_parsers  StrOutputParser
() -> AgentOutput:        ()              state[].use_sql_analyst:        ()
AgentOutput(agent_name=, findings=)                con =
duckdb.connect(knowledge_stores[])    schema_query =     schema =
con.execute(schema_query).df()    con.close()               sql_generation_prompt =
ChatPromptTemplate.from_messages([        (, ),        (, )    ])            sql_chain =
sql_generation_prompt | llm_config[] | StrOutputParser()        ()    sql_query =
sql_chain.invoke({: task_description})        sql_query = sql_query.strip().replace(,
).replace(, )    ()    :               con = duckdb.connect(knowledge_stores[])
result = con.execute(sql_query).fetchone()        patient_count = result[]  result
con.close()                        findings =         ()     Exception  e:
findings =         ()     AgentOutput(agent_name=, findings=findings)
```

The `patient_cohort_analyst` is our most advanced specialist. It's a full **Text-to-SQL** agent in a single function. The prompt engineering is the most critical part here. By providing the LLM with the exact database schema and the `Key Mappings` (e.g., how "T2DM" translates to `ICD9_CODE '25000'`).

We are giving it the precise context it needs to generate a correct and executable query. The `try...except` block is also i think is important that's why i try to use it here because it makes the agent robust by catching potential SQL errors from the LLM and preventing them from crashing the entire workflow.

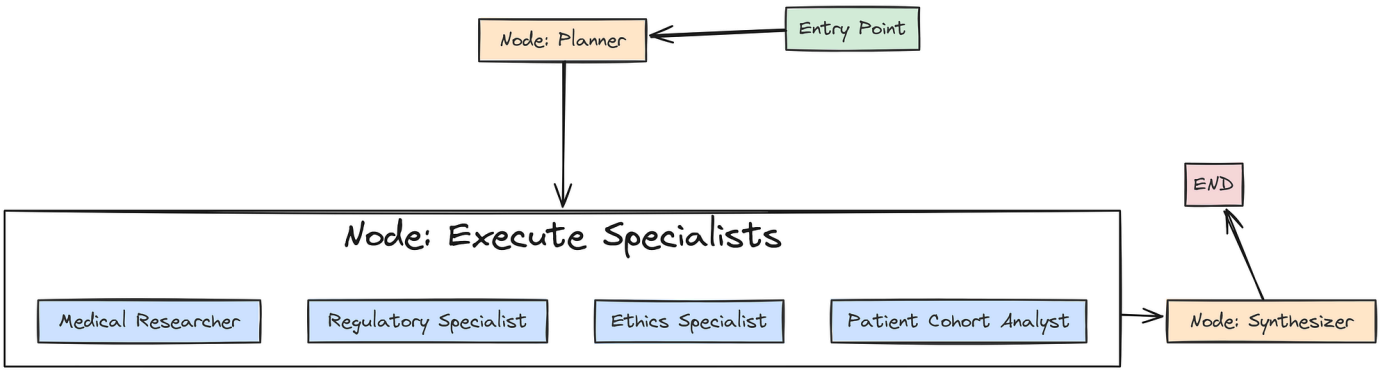
With all our data-gathering specialists defined, we need the final agent in our system: the Criteria Synthesizer. This agent's job is to act as the master writer. It will take the collected findings from all the other specialists and weave them into a single, coherent, and formally structured document.

```
 () -> GuildState:        ()            sop = state[]         drafter_llm =
ChatOllama(model=sop.synthesizer_model, temperature=)            context = .join([  out
state[]])            prompt =     ()            response = drafter_llm.invoke(prompt)
()             {**state, : response.content}
```

It aggregates all the `agent_outputs` from the state into a comprehensive **"briefing packet"**. A key feature is its dynamic model selection: `drafter_llm = ChatOllama(model=sop.synthesizer_model, ...)`. This means our AI Director can evolve the SOP to switch the synthesizer to a more powerful model (like `llama3.1:8b-instruct`) if it determines that the quality of the final draft is a key weakness. This makes the trade-off between drafting speed and quality an evolvable parameter.

## Orchestrating the Guild with LangGraph

Now that we have defined all our individual agent nodes, we can now wire them together into a collaborative workflow using `LangGraph`. We will define a graph that first calls the Planner, then executes all the specialist tasks in parallel, and finally passes their collected findings to the Synthesizer.




First, we need a special **"execution node"** that will be responsible for calling our specialist agents based on the generated plan.

```
 langgraph.graph  StateGraph, END () -> GuildState:        plan_tasks = state[][]
outputs = []             task  plan_tasks:        agent_name = task[]        task_desc =
task[]                           agent_name:            output =
retrieval_agent(task_desc, state, , )           agent_name:            output =
retrieval_agent(task_desc, state, , )           agent_name  state[].use_ethics_specialist:
output = retrieval_agent(task_desc, state, , )          agent_name:            output =
patient_cohort_analyst(task_desc, state)        :
outputs.append(output)         {**state, : outputs}
```

The `specialist_execution_node` takes the `plan` from the `GuildState` and orchestrates the execution of all the specialist tasks. The simple `if/elif` block acts as a router, dispatching each task to the correct agent function (our generic `retrieval_agent` or the specialized `patient_cohort_analyst`).

This node also demonstrates the power of our SOP feature flags: it explicitly checks `state['sop'].use_ethics_specialist` before running that agent, allowing the AI Director to dynamically enable or disable capabilities.

Now, we can finally build and compile the graph itself.

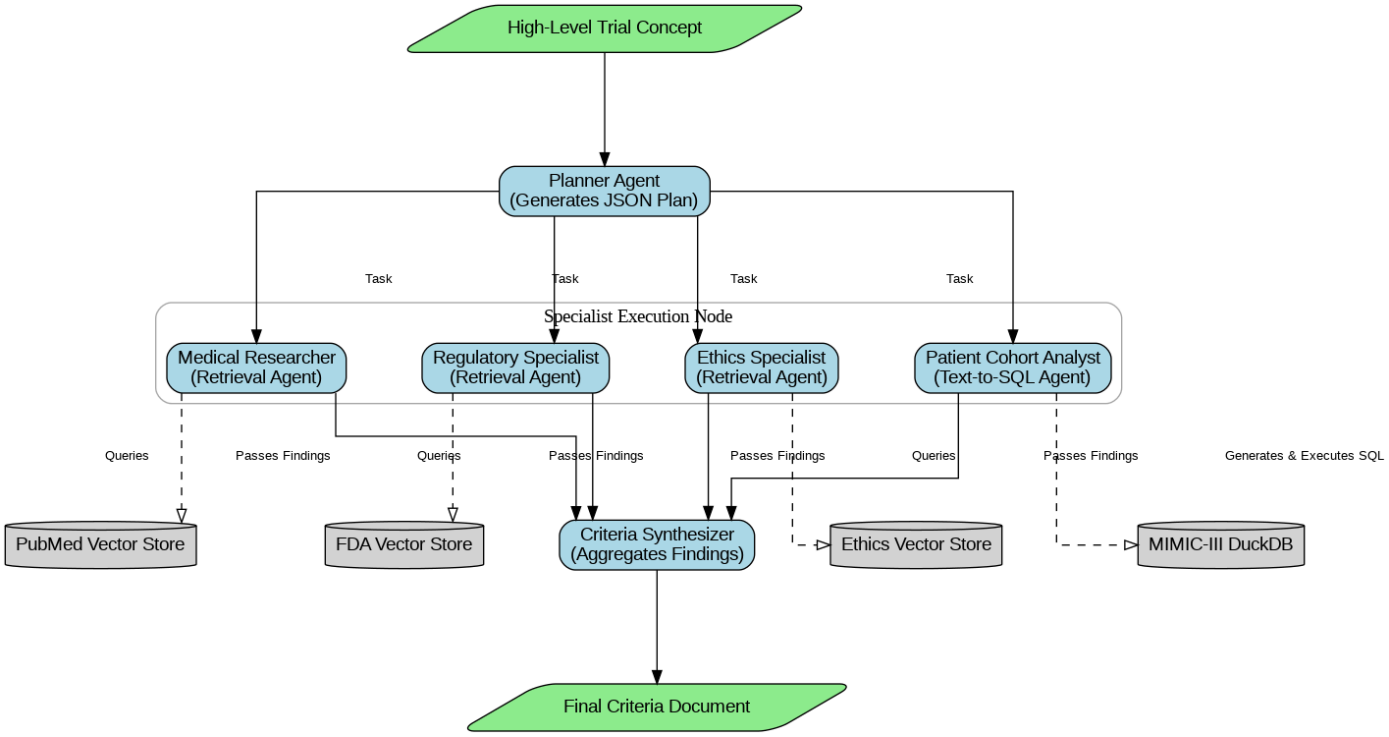
```
workflow = StateGraph(GuildState)workflow.add_node(, planner_agent)workflow.add_node(,
specialist_execution_node)workflow.add_node(,
criteria_synthesizer)workflow.set_entry_point()workflow.add_edge(, )workflow.add_edge(,
)workflow.add_edge(, END)guild_graph = workflow.()()
```

and now we assembles our final workflow. We add our three key nodes`planner`, `execute_specialists`, and `synthesizer`to the graph. Then, we use `.add_edge()` to define a simple, linear control flow: Plan -> Execute -> Synthesize. I have used the`compile()` method is the final step, transforming this flow into a fully operational `guild_graph` object that is ready to be invoked.

Let’s run this to compile the graph. We can also optionally visualize it to see the structure we’ve built.

```
:     IPython.display  Image         ImportError:    ()
```

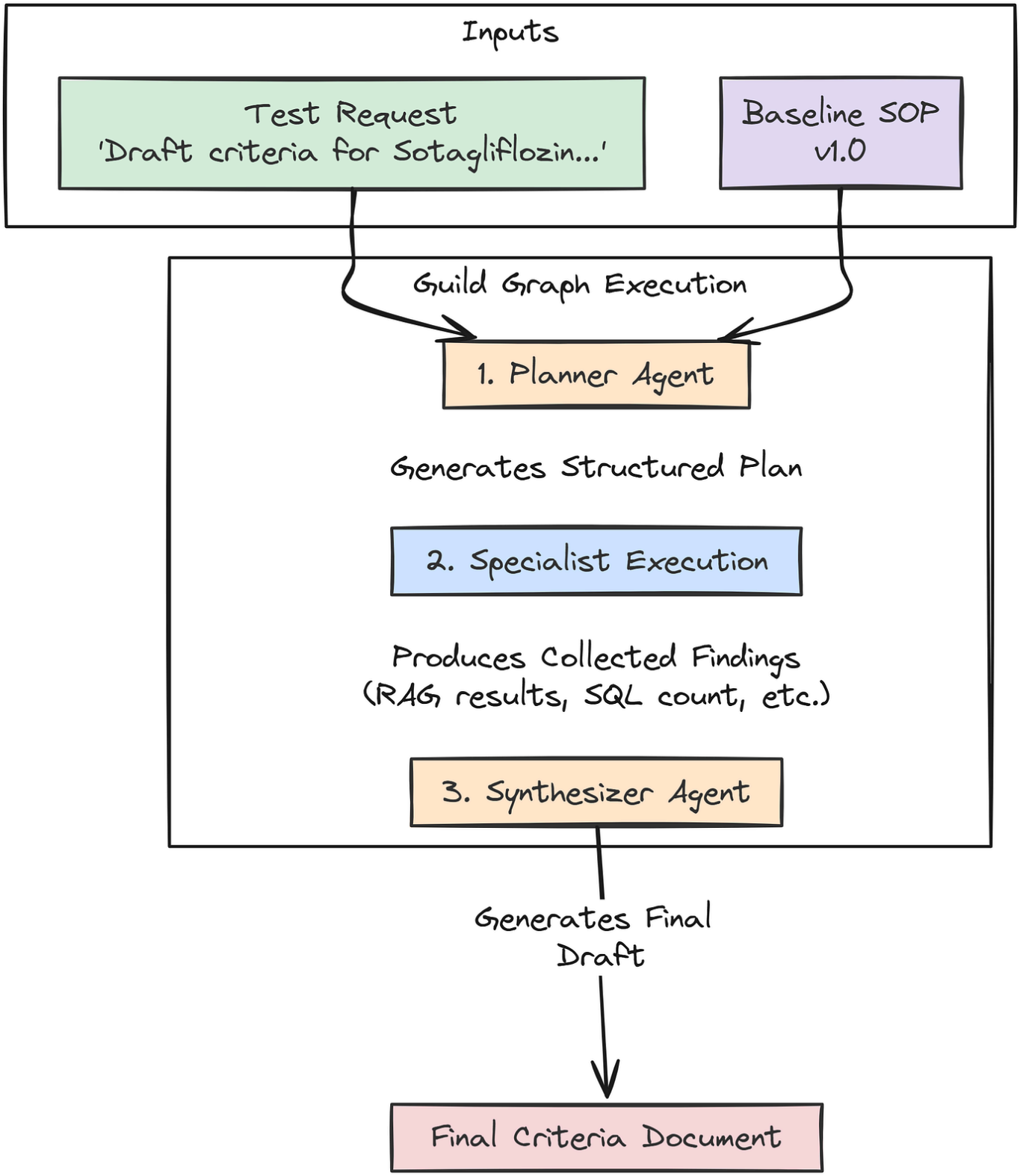
This is the graph we are getting ….




The output confirms that our `LangGraph` workflow has been successfully compiled. We now have a runnable `guild_graph` object. We have successfully built the "**Inner Loop**" of our system. It is now a fully functional, configurable, multi-agent RAG pipeline.

## Full Test Run of the Guild Graph

With the graph fully compiled, it's time to see it in action. We will conduct a full, end-to-end test run using our `baseline_sop` and a realistic trial concept. This test will validate that all our agents, data stores, and orchestration logic are working together correctly.




It will also produce our first "baseline" output, which will be the input for our evaluation and evolution loops in the subsequent parts.

```
test_request = ()graph_input = {    : test_request,    : baseline_sop}final_result =
guild_graph.invoke(graph_input)()()(final_result[])
```

Once we run this code, the `guild_graph.invoke(graph_input)` call kicks off the entire chain of events. Behind the scenes, `LangGraph` will:

1. Pass the `graph_input` to our `planner_agent`.
2. Take the planner's output and pass the updated state to the `specialist_execution_node`.
3. The execution node will call all our specialists in turn.
4. Finally, the state, now rich with findings, will be passed to the `criteria_synthesizer` to produce the final document.

Let's run it and observe the detailed logs from each agent as it executes.

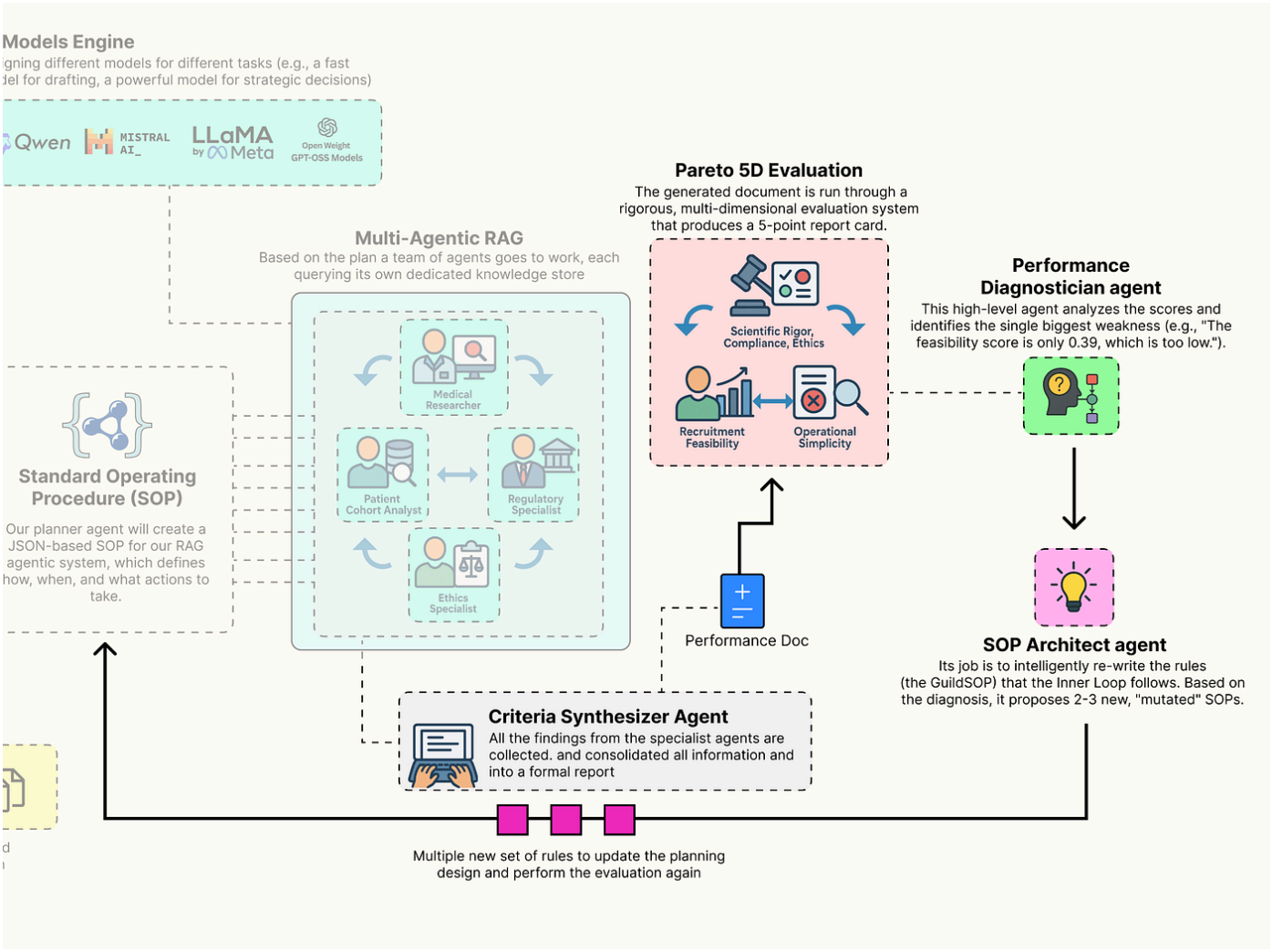
```
Running the full Guild graph with baseline SOP v1.0...Generated Plan:{  : [    { : , : , : [] },    { : , : , : [] },    { : , : , : [] },    { : , : , : [] }  ]}Retrieved 3 documents....Using k=3  retrieval.Retrieved 3 documents....Retrieved 2 documents....Generated SQL Query:SELECT COUNT(DISTINCT p.subject_id)FROM patients p ...Query executed successfully. Estimated patient count: 59Synthesizer is using model .Final criteria generated.----------**Inclusion Criteria:**1. Male or female adults, age 18 years or older.2. Diagnosis of Type 2 Diabetes Mellitus (T2DM).3. Uncontrolled T2DM, defined as a Hemoglobin A1c (HbA1c) value > 8.0% at screening....**Exclusion Criteria:**1. Diagnosis of Type 1 Diabetes Mellitus.2. History of severe hypoglycemia within the past 6 months....
```

We can see a step-by-step trace of our Guild's collaborative process. We can see the Planner creating a logical plan, each specialist executing its task by accessing the correct knowledge store (with the Cohort Analyst even generating and running a complex SQL query), and finally, the Synthesizer assembling all the findings into a well-structured document.

We have now built and successfully tested a complete, multi-agent RAG pipeline using real-world data sources. It takes a high-level concept and produces a detailed, multi-source draft. The next, and most crucial, part is to build the system that evaluates and improves this Guild.

## Multi-Dimensional Evaluation System

A self-improving system is only as good as its ability to measure its own performance. We have built a system that can produce a detailed document, but how do we know if that document is good? And more importantly, how can our AI Research Director learn to make it better?



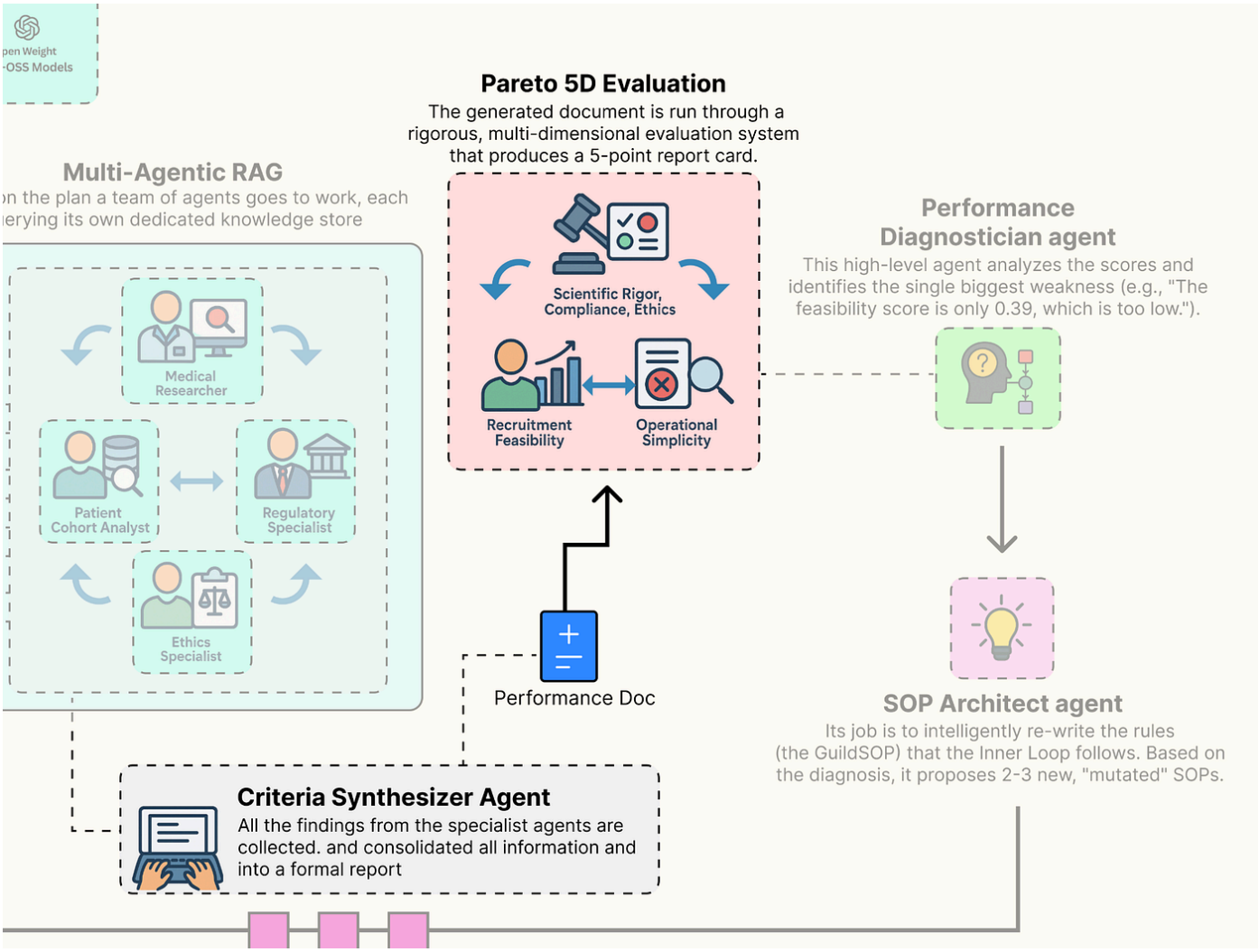

To do this, we need to move beyond simplistic, single-score metrics like accuracy. The quality of a clinical trial protocol is multi-dimensional. We will now build a sophisticated evaluation suite that is going to measure the Guild output across the five competing pillars we identified at the start. This gauntlet will provide the rich, multi-dimensional feedback signal that is the lifeblood of our evolutionary outer loop.

In this section, here's what we are going to do:

- We will build three separate evaluators using our most powerful model (`llama3:70b`) to act as expert judges for the qualitative aspects of Scientific Rigor, Regulatory Compliance, and Ethical Soundness.
- We will write two fast, reliable, and objective programmatic functions to score the quantitative aspects of Recruitment Feasibility and Operational Simplicity.
- Wrapping all five of these individual evaluators into a single, master function that takes the final output of our Guild and generates the 5D performance vector our AI Director will use to make its decisions.

## Building a Custom Evaluator for Each Parameter

We will define each of our five evaluators as a separate, specialized function. This approach allows us to fine-tune the logic for each dimension of quality independently.




First, a small utility: we will define a Pydantic model to ensure the output of our LLM judges is always structured, containing both a numerical score and a textual justification.

```
 langchain_core.pydantic_v1  BaseModel, Field ():           score:  = Field(description=)
reasoning:  = Field(description=)
```

This `GradedScore` class is a simple piece of engineering. By forcing our evaluator LLMs to return their feedback in this specific JSON format, we make the results reliable and easy to parse. We can now count on always receiving a numerical `score` and a `reasoning` string, which makes our entire evaluation and evolution system more robust.

Now, let's build our first LLM-as-a-Judge, focused on Scientific Rigor.

```
 langchain_core.prompts  ChatPromptTemplate () -> GradedScore:
evaluator_llm = llm_config[].with_structured_output(GradedScore)            prompt =
ChatPromptTemplate.from_messages([        (, ),                (, )    ])           chain
= prompt | evaluator_llm         chain.invoke({: generated_criteria, : pubmed_context})
```

The `scientific_rigor_evaluator` function is our first expert judge. It takes the final `generated_criteria` and the specific `pubmed_context` that the Medical Researcher agent found. By providing both to the evaluator LLM, we are asking a very specific question: "Is this output grounded in this evidence?" This is our primary defense against hallucination and ensures that the Guild's proposals are scientifically sound.

Next, we will build the judge responsible for Regulatory Compliance.

```
 () -> GradedScore:        evaluator_llm =
llm_config[].with_structured_output(GradedScore)        prompt =
ChatPromptTemplate.from_messages([        (, ),        (, )    ])    chain = prompt |
evaluator_llm         chain.invoke({: generated_criteria, : fda_context})
```

This `regulatory_compliance_evaluator` function is another specialized judge. Its sole focus is to compare the generated criteria against the `fda_context`. By creating separate, focused evaluators for each knowledge domain, we get much more targeted and reliable feedback. This is a far better approach than asking a single, generic evaluator to judge everything at once.

Our third LLM judge will measure the Ethical Soundness.

```
 () -> GradedScore:        evaluator_llm =
llm_config[].with_structured_output(GradedScore)        prompt =
ChatPromptTemplate.from_messages([        (, ),        (, )    ])    chain = prompt |
evaluator_llm         chain.invoke({: generated_criteria, : ethics_context})
```

The `ethical_soundness_evaluator` completes our trio of LLM-as-a-Judge specialists. It ensures that our system's output is not just scientifically and legally sound, but also ethically responsible. This is a critical component for any real-world medical AI application.

Now, we will move on to our programmatic evaluators. Not all metrics require the nuanced reasoning of an LLM. For objective, quantifiable aspects, simple Python functions are faster, cheaper, and 100% reliable. Let’s build the evaluator for Recruitment Feasibility.

```
 () -> GradedScore:            findings_text = cohort_analyst_output.findings    :
count_str = findings_text.split()[].replace(, )        patient_count = (count_str)
(IndexError, ValueError):                GradedScore(score=, reasoning=)
IDEAL_COUNT =         score = (, patient_count / IDEAL_COUNT)    reasoning =
GradedScore(score=score, reasoning=reasoning)
```

It doesn't need an LLM because the evaluation is purely mathematical. It takes the structured output from our `Patient Cohort Analyst`, parses the estimated patient count, and normalizes it to a 0-1 score. This function provides a hard, data-driven feedback signal. If the generated criteria

are too strict, the patient count will be low, and this score will be low, telling our AI Director that a change is needed.

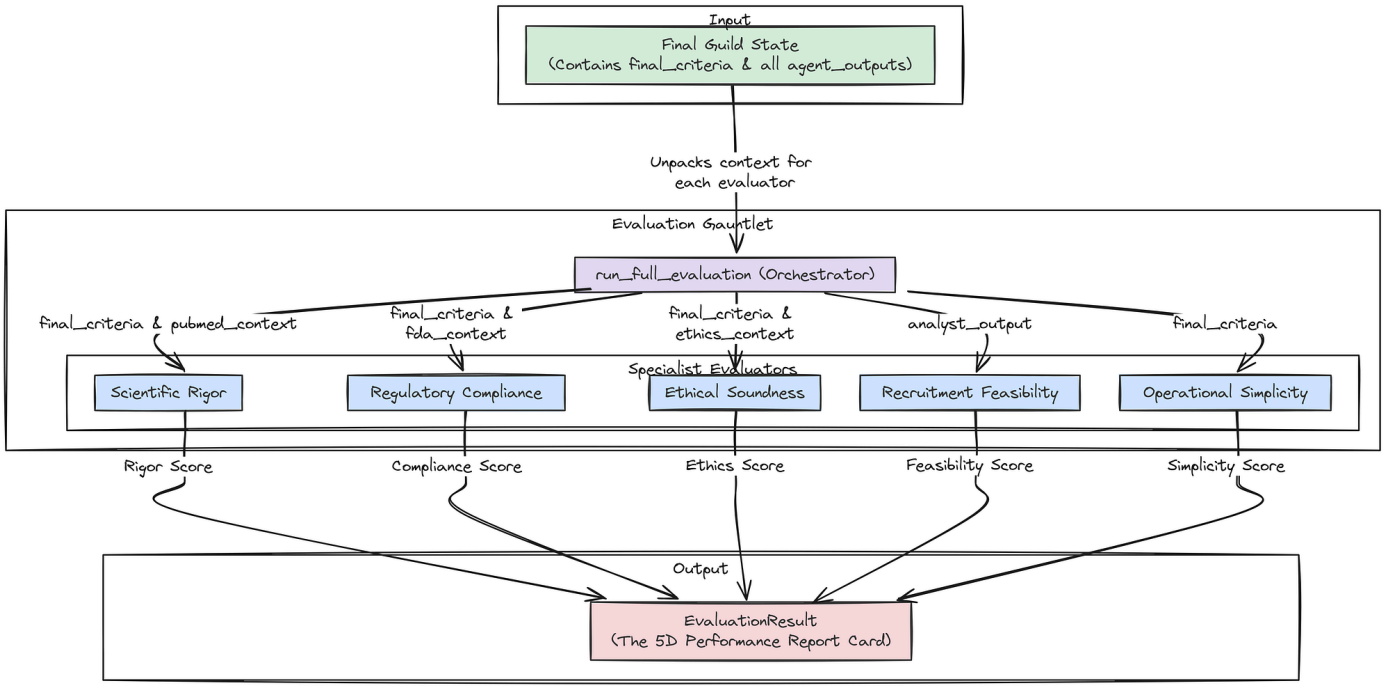
Our final evaluator will be another programmatic one, scoring Operational Simplicity.

```
 () -> GradedScore:            EXPENSIVE_TESTS = [, , , , , ]           test_count = (
test  EXPENSIVE_TESTS  test  generated_criteria.lower())          score = (,  -
(test_count * ))    reasoning =      GradedScore(score=score, reasoning=reasoning)
```

The `simplicity_evaluator` is a simple but effective heuristic for estimating operational cost. It acts as a **"red flag"** system. By scanning for keywords related to expensive procedures, it provides a penalty for criteria that might be scientifically sound but impractical to implement on a large scale. This provides another crucial, real-world constraint for our optimization problem.

## Creating the Aggregate LangSmith Evaluator

Now that we have our five specialist evaluators, we need to wrap them into a single, master function. This aggregate function will orchestrate the entire evaluation system, taking the final state of the Guild graph and returning the complete 5D performance vector that our AI Research Director will use to make its decisions.




First, let's define the Pydantic model for the final, aggregated result.

```
 ():        rigor: GradedScore    compliance: GradedScore    ethics: GradedScore
feasibility: GradedScore    simplicity: GradedScore
```

This `EvaluationResult` class is the final data product of our evaluation gauntlet. It neatly packages the `GradedScore` from each of our five pillars into a single, structured object.

Now, we can build the master `run_full_evaluation` function.

```
 () -> EvaluationResult:        ()            final_criteria = guild_final_state[]
agent_outputs = guild_final_state[]               pubmed_context = ((o.findings  o
agent_outputs  o.agent_name == ), )    fda_context = ((o.findings  o  agent_outputs
o.agent_name == ), )    ethics_context = ((o.findings  o  agent_outputs  o.agent_name ==
), )    analyst_output = ((o  o  agent_outputs  o.agent_name == ), )            ()
rigor = scientific_rigor_evaluator(final_criteria, pubmed_context)    ()    compliance =
regulatory_compliance_evaluator(final_criteria, fda_context)    ()    ethics =
ethical_soundness_evaluator(final_criteria, ethics_context)    ()    feasibility =
feasibility_evaluator(analyst_output)  analyst_output  GradedScore(score=, reasoning=)
()    simplicity = simplicity_evaluator(final_criteria)        ()
EvaluationResult(rigor=rigor, compliance=compliance, ethics=ethics,
feasibility=feasibility, simplicity=simplicity)
```

The `run_full_evaluation` function is the conductor of our evaluation orchestra. It takes the `guild_final_state`, which contains all the artifacts from the Guild's run, and carefully unpacks it. It intelligently routes the correct pieces of context (e.g., `pubmed_context`) to the correct evaluators (e.g., `scientific_rigor_evaluator`).

This function is the final step in our **"Inner Loop"**, transforming the Guild's raw text output into the structured, multi-dimensional performance vector that the "Outer Loop" needs to begin the process of evolution.

Let’s run our new evaluation gauntlet on the output we generated from our baseline SOP run in earlier part.

```
baseline_evaluation_result = run_full_evaluation(final_result)()
(json.dumps(baseline_evaluation_result.(), indent=))
```

This is the output we are getting …

```
--- RUNNING FULL EVALUATION GAUNTLET ---Evaluating: Scientific Rigor...Evaluating:
Regulatory Compliance...Evaluating: Ethical Soundness...Evaluating: Recruitment
Feasibility...Evaluating: Operational Simplicity...--- EVALUATION GAUNTLET COMPLETE ---
Full Evaluation Result  Baseline SOP:{    : {        : 0.9,        :     },    : {
: 0.95,        :     },    : {        : 1.0,        :     },    : {        :
0.3933333333333333,        :     },    : {        : 1.0,        :     }}
```

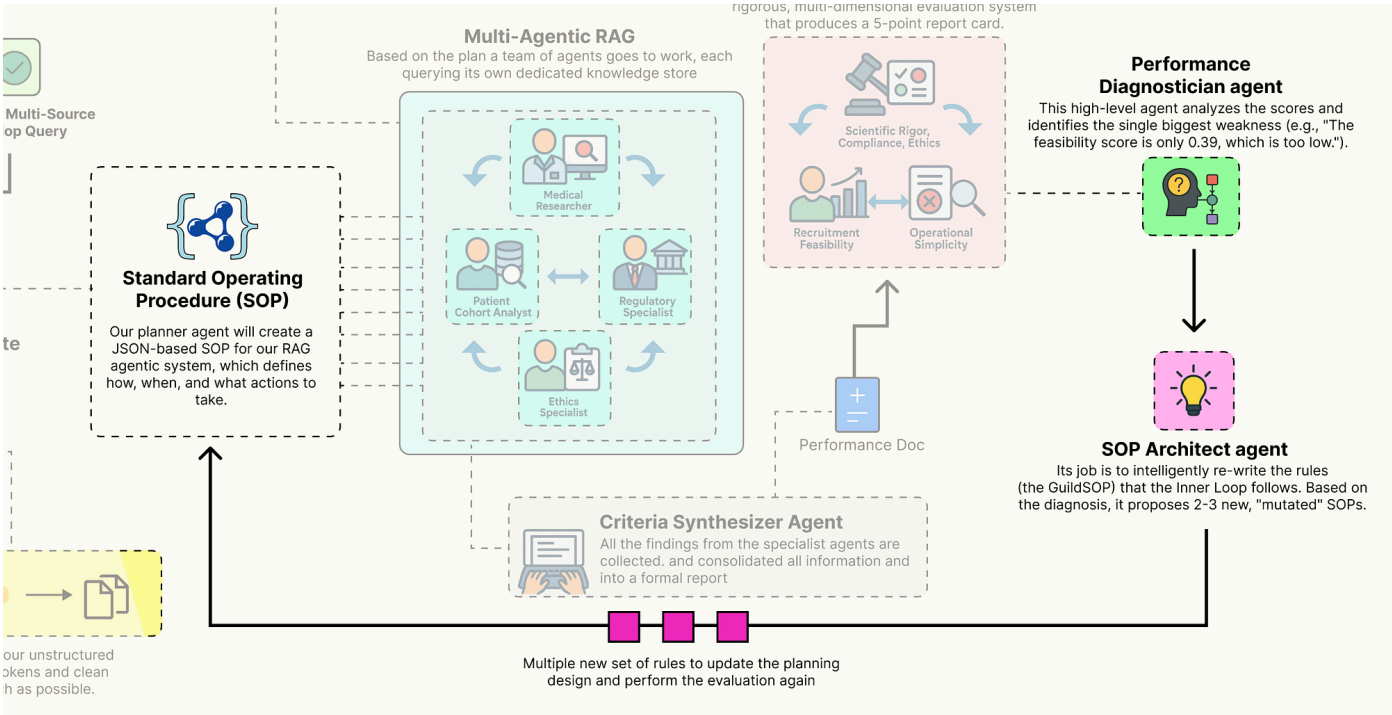
This structured output is the **“performance report card”** for our baseline SOP, and it is some important info. It tells a clear story: our initial, hand-engineered process is very good at creating criteria that are scientifically rigorous (0.9), compliant (0.95), ethical (1.0), and simple (1.0).

However, it reveals a critical weakness: **Recruitment Feasibility**. A score of just 0.39 means that while the protocol is **“good”** on paper, it would likely fail in the real world because it would be nearly impossible to find enough patients.

This is the precise, actionable, multi-dimensional feedback our AI Research Director needs. It has not just been told the output is **“bad”**, it has been told exactly which dimension is failing and why. The stage is now perfectly set for next part, where the Director will analyze this very report and attempt to evolve the SOP to fix this specific feasibility problem.

## Outer Loop of the Evolution Engine

We have successfully built and evaluated our architecture. We have a system that can produce a high-quality draft, and we have an evaluation component that provides a rich, 5D performance vector. But so far, the process is static. The Guild will produce the same output for the same input, with the same weaknesses, every time.




Now, we are going to build the brain of our self-improving system: the **“AI Research Director”.** This is our **“Outer Loop”**. It’s a higher-level agentic system whose job is not to design clinical trials, but to improve the process of designing clinical trials.

It will analyze the 5D performance vector from our evaluation gauntlet, diagnose the root cause of any weaknesses, and intelligently rewrite the Guild’s own `GuildSOP` to address them. This is where we implement the core evolutionary concepts that allow our system to learn and adapt.

In this section, here’s what we are going to do:

- We will build a simple class to store and manage our evolving SOPs and their performance scores, creating a of process configurations.
- We will implement the two core agents of the Director: the `Performance Diagnostician`, which identifies weaknesses, and the `SOP Architect`, which proposes solutions.

- Then define a master function that orchestrates a single, complete : Diagnose -> Evolve -> Evaluate.
- Going to execute this loop to show the system autonomously identifying the feasibility weakness in our baseline SOP and generating new, mutated SOPs to try and fix it.

## Managing Guild Configurations

Before we can evolve our SOPs, we need a place to store them. We will create a simple class that will do that. This class will keep track of every version of the `GuildSOP` that our system generates, along with its corresponding 5D evaluation result and its lineage (which parent version it evolved from). This provides a complete, traceable history of our evolutionary process.

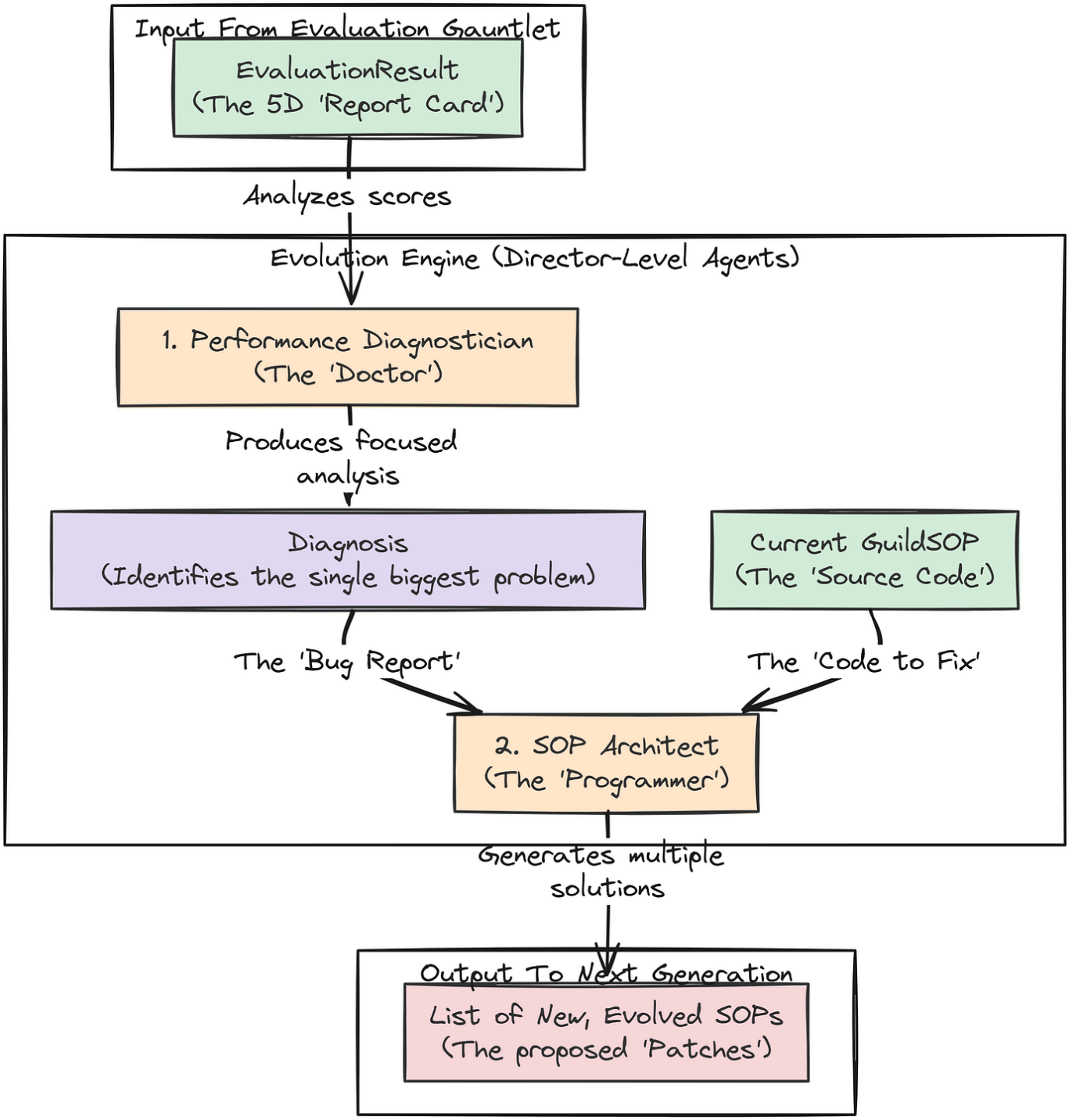
```
 :         ():                self.pool: [[, ]] = []                self.version_counter =
():                self.version_counter +=         entry = {            :
self.version_counter,            : sop,            : eval_result,            :
parent_version         }        self.pool.append(entry)        ()             () -> [[,
]]:                self.pool[-]  self.pool
```

The `SOPGenePool` class is a straightforward but important data management tool. It's our lab notebook for the evolutionary process. The `add` method is the key function, cataloging each new `GuildSOP` with its performance data. By storing the `parent_version`, we create a clear chain of ancestry.

This will allow us to later trace back a highly successful SOP and understand the sequence of mutations that led to its discovery. It's a simple implementation of a version control system for our agent's own "source code."

## Building The Director-Level Agents

Now we define the two agents that form the core of our evolution engine. These agents operate at a higher level of abstraction. They don’t reason about medicine or regulations, they reason about process and performance.




First up is the **Performance Diagnostician**. This agent's job is to look at the 5D performance vector from the evaluation gauntlet and identify the single biggest problem.

```
 ():            primary_weakness: [, , , , ]        root_cause_analysis:  =
Field(description=)        recommendation:  = Field(description=) () -> Diagnosis:
()        diagnostician_llm = llm_config[].with_structured_output(Diagnosis)
prompt = ChatPromptTemplate.from_messages([        (, ),        (, )    ])        chain =
prompt | diagnostician_llm         chain.invoke({: eval_result.json()})
```

Let's try to understand our first agent …

1. The `performance_diagnostician` agent is the for our RAG pipeline. It takes the `EvaluationResult` (the "symptoms") and produces a structured `Diagnosis`.
2. By forcing it to identify a `primary_weakness` from a `Literal` set and provide a `root_cause_analysis`, we are guiding it to perform a focused, analytical task. Its output isn't just a complaint; it's an actionable insight that will directly inform the next agent in our evolutionary loop.

The second agent is the **SOP Architect**. This agent is the **evolver**, It takes the diagnosis from the previous step and the current `GuildSOP`, and its job is to generate several new, mutated versions of the SOP, each representing a different strategy to solve the identified problem.

```
 ():        mutations: [GuildSOP] () -> EvolvedSOPs:        ()        architect_llm =
llm_config[].with_structured_output(EvolvedSOPs)                prompt =
ChatPromptTemplate.from_messages([        (, ),        (, )    ])        chain = prompt |
architect_llm     chain.invoke({: current_sop.json(), : diagnosis.json()})
```

1. The `sop_architect` is the creative engine of our self-improving system. Its prompt is a kind of instruction engineering. We are telling the LLM: **`current_sopdiagnosismutations`**.
2. By providing the `GuildSOP.schema_json()` directly in the prompt, we drastically increase the likelihood that the LLM will generate valid, correctly formatted new SOPs. This agent doesn't just randomly change things; it proposes targeted, intelligent modifications based on the specific problem identified by the diagnostician.

## Running The Full Evolutionary Loop

We now have all the components for a single **generation** of evolution, a gene pool to store our results, a diagnostician to identify problems, and an architect to propose solutions. We can now wrap these into a master function that orchestrates one full cycle of Diagnose -> Evolve -> Evaluate.

```
 ():        ( + * +  + *)            current_best_entry = gene_pool.get_latest_entry()
parent_sop = current_best_entry[]    parent_eval = current_best_entry[]    parent_version
= current_best_entry[]    ()            diagnosis = performance_diagnostician(parent_eval)
()        new_sop_candidates = sop_architect(diagnosis, parent_sop)    ()         i,
candidate_sop  (new_sop_candidates.mutations):        ()                guild_input = {:
trial_request, : candidate_sop}        final_state = guild_graph.invoke(guild_input)
eval_result = run_full_evaluation(final_state)
gene_pool.add(sop=candidate_sop, eval_result=eval_result, parent_version=parent_version)
( + * +  + *)
```

The `run_evolution_cycle` function is the main orchestrator of our Outer Loop. It formalizes the **genetic algorithm** process. It takes the best-performing SOP from the previous generation, uses the Director-level agents to diagnose its flaws and architect potential improvements, and then rigorously tests each of those new **"child"** SOPs by running them through the full inner loop and evaluation gauntlet. This function represents one complete turn of our system's self-improvement flywheel.

Let's put it all together. We will initialize our `SOPGenePool`, add our baseline SOP and its evaluation result, and then run a single evolution cycle.

```
gene_pool = SOPGenePool()()gene_pool.add(sop=baseline_sop,
eval_result=baseline_evaluation_result)run_evolution_cycle(gene_pool, test_request)
```

So, when we run this, we got the following output ….

```
Initialized SOP Gene Pool.Added SOP v1 to the gene pool.Improving upon SOP v1...Diagnosis
complete. Primary Weakness: . Recommendation: The primary goal should be to modify the SOP
to increase the estimated patient count...Generated 2 new SOP candidates....Query executed
successfully. Estimated patient count: 121...Added SOP v2 to the gene pool....Using k=5
retrieval....Added SOP v3 to the gene pool.
```

The output is a showing the working of our autonomous system …

1. The `Performance Diagnostician` correctly analyzed the evaluation of SOP v1 and identified as the primary weakness.
2. The `SOP Architect` took this diagnosis and generated two new, targeted mutations to try and solve the problem.
3. The system then rigorously tested each of these new candidates (SOP v2 and SOP v3), running the full inner loop and evaluation system for both.
4. Finally, both new SOPs and their performance results were added to our `SOPGenePool`.

The process worked exactly as designed. The system has autonomously identified a problem and generated and tested potential solutions. The next step is to analyze the results of this cycle to see if the proposed mutations were successful.

## 5D Pareto Based Analysis

Our evolutionary loop has completed a full cycle. It has diagnosed the weakness in our baseline SOP and generated and tested two new **"mutant"** SOPs designed to fix the problem. The `SOPGenePool` now contains three distinct process configurations, each with a complete 5D performance vector.

Now comes the final step: analyzing these results to make an decision. In a multi-objective optimization problem, there is often no single **"best"** solution. Instead, there is a set of optimal trade-offs, known as the **Pareto Frontier**. Our goal is to identify this frontier and present it to a human decision-maker.

In this final section, here's what we are going to do:

- We will first print a summary of all the SOPs and their performance scores to see the direct impact of the mutations.
- We will write a function to programmatically identify the non-dominated solutions in our gene pool the set of SOPs that represent the best possible trade-offs.

- Create a powerful visualization, a , that allows us to see the performance of our optimal SOPs across all five dimensions simultaneously, making the trade-offs clear and intuitive.

First, let's just print out the scores for all the SOPs currently in our gene pool to get a high-level overview of what happened.

```
()() entry  gene_pool.pool:    v = entry[]    p = entry[]    evals = entry[]        r, c,
e, f, s = evals.rigor.score, evals.compliance.score, evals.ethics.score,
evals.feasibility.score, evals.simplicity.score    parent_str =   p        ()
```

When we run this code this is the overall performance we are getting …

```
SOP Gene Pool Evaluation Summary:---------------------------------SOP v1 (Parent)     :
Rigor=0.90, Compliance=0.95, Ethics=1.00, Feasibility=0.39, Simplicity=1.00SOP v2 (Child
of v1): Rigor=0.85, Compliance=0.95, Ethics=1.00, Feasibility=0.81, Simplicity=1.00SOP v3
(Child of v1): Rigor=0.90, Compliance=0.95, Ethics=1.00, Feasibility=0.39, Simplicity=1.00
```

This summary table tells some analysis. It is the direct evidence that our autonomous system worked.

- Our AI Director correctly identified that had a `Feasibility` score of .
- It then generated , a mutation designed to fix this. The result is a massive success: the `Feasibility` score more than doubled to ! This came at the cost of a small, acceptable decrease in `Rigor` (from 0.90 to 0.85), demonstrating an intelligent trade-off.
- It also generated , which tried a different strategy (increasing the `k` for the researcher). This had on feasibility, showing that the system is capable of exploring different paths, not all of which are successful.

We have successfully created a system that can reason about its own failures and intelligently rewrite its internal processes to improve.

## Identifying the Pareto Front

Now, we need to formalize the concept of an **"optimal trade-off"**. In our gene pool, some solutions might be strictly worse than others. For example, SOP v3 has the same scores as SOP v1 on four metrics and is equal on feasibility. There's no reason to ever choose v3. We say that v3 is "dominated" by v1.

The **Pareto Front** is the set of all non-dominated solutions. We'll write a function to identify this set from our gene pool.

```
 numpy  np () -> [[, ]]:        pareto_front = []    pool_entries = gene_pool.pool
i, candidate  (pool_entries):        is_dominated =                 cand_scores =
np.array([s[]  s  candidate[].().values()])                 j, other  (pool_entries):
i == j:                          other_scores = np.array([s[]  s  other[].().values()])
np.(other_scores >= cand_scores)  np.(other_scores > cand_scores):
is_dominated =                                          is_dominated:
pareto_front.append(candidate)                pareto_front
```

The `identify_pareto_front` function is a classic implementation of a Pareto dominance check. It's a brute-force but effective algorithm that systematically compares each SOP's 5D performance vector against every other SOP's vector. The logic `np.all(other_scores >= cand_scores) and np.any(other_scores > cand_scores)` is the formal mathematical definition of Pareto dominance. This function will distill our entire gene pool down to only the most rational, optimal choices.

Let’s run it on our pool and see which SOPs make the cut.

```
pareto_sops = identify_pareto_front(gene_pool)()() entry  pareto_sops:    v = entry[]
evals = entry[]    r, c, e, f, s = evals.rigor.score, evals.compliance.score,
evals.ethics.score, evals.feasibility.score, evals.simplicity.score    ()

SOPs on the Pareto Front:-------------------------SOP v1: Rigor=0.90, Compliance=0.95,
Ethics=1.00, Feasibility=0.39, Simplicity=1.00SOP v2: Rigor=0.85, Compliance=0.95,
Ethics=1.00, Feasibility=0.81, Simplicity=1.00
```

The algorithm has correctly identified that **SOPs v1 and v2** form our Pareto Front. SOP v3 was correctly eliminated because it is dominated by SOP v1. This is the final, distilled output of our entire system. It doesn’t give us a single **“best”** answer. Instead, it presents a human decision-maker with a menu of optimal, but different, strategies:

- The strategy, prioritizing scientific purity at the cost of low recruitment feasibility.
- The strategy, which makes a small sacrifice in rigor to achieve a massive gain in real-world practicality.

The final choice between these two is a strategic business decision, not a purely technical one. Our job is complete: it has found and presented the best possible trade-offs.

## Visualizing the Frontier & Making a Decision

Visualizing a 5-dimensional space is impossible. However, there are techniques for showing high-dimensional trade-offs. One of the best is the **parallel coordinates plot**. This plot draws each of our SOPs as a line, with each vertical axis representing one of our five performance pillars. It allows us to instantly see how each strategy performs across all dimensions and where the trade-offs lie.

We will write a function to generate this plot, along with a simpler 2D scatter plot focusing on the main Rigor vs. Feasibility trade-off we discovered.

```
 matplotlib.pyplot  plt pandas  pd ():          pareto_sops:        ()                fig,
(ax1, ax2) = plt.subplots(, , figsize=(, ))           labels = [  s  pareto_sops]
rigor_scores = [s[].rigor.score  s  pareto_sops]    feasibility_scores =
[s[].feasibility.score  s  pareto_sops]        ax1.scatter(rigor_scores,
feasibility_scores, s=, alpha=, c=)     i, txt  (labels):        ax1.annotate(txt,
(rigor_scores[i], feasibility_scores[i]), xytext=(,-), textcoords=, fontsize=)
ax1.set_title(, fontsize=)    ax1.set_xlabel(, fontsize=)    ax1.set_ylabel(, fontsize=)
ax1.grid(, linestyle=, alpha=)    ax1.set_xlim((rigor_scores)-, (rigor_scores)+)
ax1.set_ylim((feasibility_scores)-, (feasibility_scores)+)        data = []     s
pareto_sops:        eval_dict = s[].()        scores = {k.capitalize(): v[]  k, v
eval_dict.items()}        scores[] =         data.append(scores)        df =
pd.DataFrame(data)        pd.plotting.parallel_coordinates(df, , colormap=plt.get_cmap(),
ax=ax2, axvlines_kwargs={: , : })    ax2.set_title(, fontsize=)    ax2.grid(, which=,
axis=, linestyle=, alpha=)    ax2.set_ylabel(, fontsize=)    ax2.legend(loc=,
bbox_to_anchor=(, -), ncol=(labels))    plt.tight_layout()    plt.show()
```

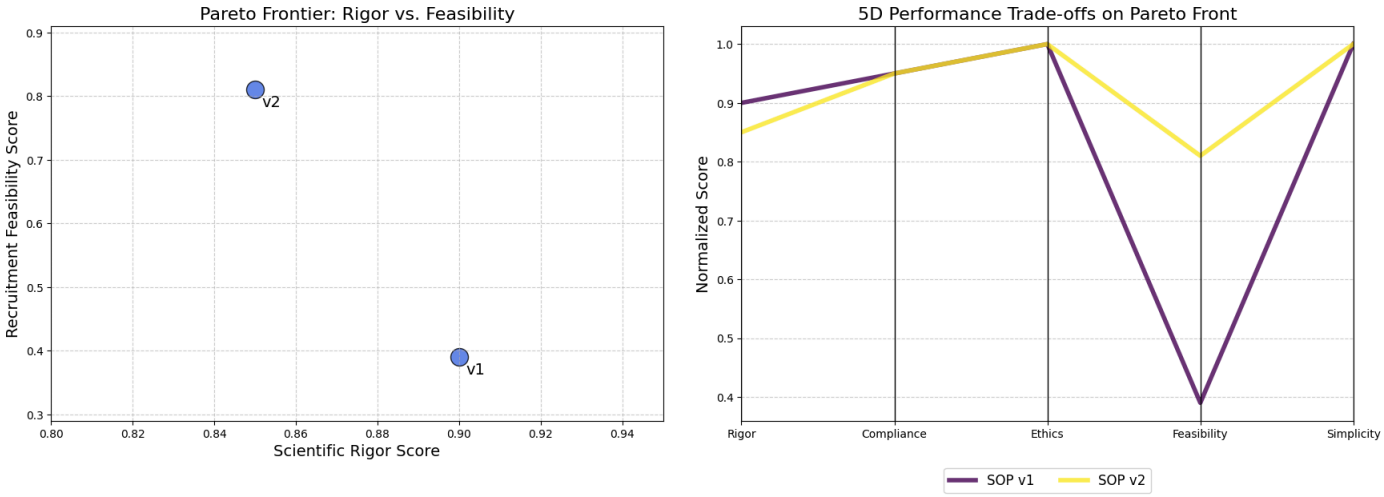
This `visualize_frontier` function is our final reporting tool. It takes the list of optimal `pareto_sops` and creates two powerful visualizations. The scatter plot provides a classic view of the two-dimensional trade-off between our most conflicting objectives.

The parallel coordinates plot is the main key, it displays the full 5D performance profile of each optimal SOP, allowing a human decision-maker to see the complete picture at a glance.

Let’s run the visualization on our identified Pareto front.

```
visualize_frontier(pareto_sops)
```

This final visualization is the ultimate output of our entire system. It’s not just an answer, it’s a decision-support tool.




- clearly shows the trade-off. To move from to , we must accept a small decrease in to achieve a large gain in .

- tells the full story. We can trace the lines for and . We see that they are identical on the “Compliance,” “Ethics,” and “Simplicity” axes. The lines only diverge on “Rigor” and “Feasibility.” The “crossing” pattern between these two axes is the classic visual signature of a trade-off. A user can instantly see that v2’s line is much higher on Feasibility, while v1’s line is slightly higher on Rigor.

This visualization gives info to a human expert. Instead of trusting a black box, they can see the optimal strategies the AI has discovered and make a final, informed decision based on their own priorities.

If the trial is a high-risk, exploratory study where scientific purity is paramount, they might choose **v1**. If it’s a later-stage trial where rapid recruitment is the top priority, they would almost certainly choose **v2**.

## Understanding the Cognitive Workflow

We have successfully built a system that evolves and improves its own processes. We’ve seen the high-level results in the gene pool summary and the Pareto front. But what does a single, high-performing run actually *look like* on the inside? How do the agents collaborate? Where is the time spent? How do the final performance scores translate into a visual profile?
To answer these questions, we need to move from the macro-view of evolution to the micro-view of a single cognitive cycle. In this section, we will conduct a forensic analysis of one complete run of our improved `SOP v2`. We will turn the Guild from a "black box" into a "glass box," observing its internal mechanics in detail.

Here’s what we are going to do:

- We will create a new invocation function that precisely measures the start time, end time, and duration of each agent’s execution within the `LangGraph`.
- We will use this timing data to generate a Gantt chart, providing a clear and intuitive visualization of the Guild’s workflow, highlighting both parallel and sequential operations.
- We will create a Radar Chart to visualize the 5D performance vector of our baseline and improved SOPs, making the multi-objective trade-offs immediately apparent.

### Visualizing the Agentic Workflow Timeline

First, we want to understand the *process* of the Guild’s collaboration. Is it truly parallel? Which agent is the bottleneck? To find out, we need to instrument our graph execution to capture timing information for each node.

We will write a new wrapper function, `invoke_with_timing`, that uses the `.stream()` method of our compiled graph. The `stream()` method is a powerful feature that yields the output of each node as it completes. By recording the timestamp before and after each node's execution, we can capture the raw data needed to build a timeline.

```
 time collections  defaultdict ():        ()            timing_data = []
start_times = defaultdict()            graph_input = {: request, : sop}             event
graph.stream(graph_input, stream_mode=):                node_name = (event.keys())[]
end_time = time.time()                                        node_name   start_times:
start_times[node_name] = end_time -                   start_time = end_time - duration
timing_data.append({            : node_name,            : start_time,            :
end_time,            : duration        })        start_times[node_name] = start_time
overall_start_time = (d[]  d  timing_data)     data  timing_data:        data[] -=
overall_start_time        data[] -= overall_start_time               final_state =
event[(event.keys())[-]]     final_state, timing_data
```

The `invoke_with_timing` function is our instrumentation layer. It wraps the standard `.stream()` call and adds performance monitoring. For each event yielded by the stream, it captures the node name and timestamps.

Let's run this function with our successful `SOP v2` to capture its performance profile.

```
sop_v2 = gene_pool.pool[][]final_state_v2, timing_data_v2 =
invoke_with_timing(guild_graph, sop_v2, test_request)()(json.dumps(timing_data_v2,
indent=))
```

Let's run this code and see the duration progress of our SOP v2 …

```
--- Instrumenting Graph Run  SOP: {: , : 3, ...} ---......[  {    : ,    : 0.0,    : 5.2,
: 5.2  },  {    : ,    : 5.2,    : 20.7,    : 15.5  },  {    : ,    : 20.7,    : 25.5,
: 4.8  }]
```

The output shows that we have successfully captured the execution time for each major stage of our Guild's workflow. We have the raw data: the `planner` took 5.2 seconds, the `execute_specialists` node took 15.5 seconds, and the `synthesizer` took 4.8 seconds. This data is useful, but it's not intuitive. We now need to visualize it.

Now, we will write a function to take this timing data and plot it as a Gantt chart. This will give us an immediate, visual understanding of the workflow's timeline.

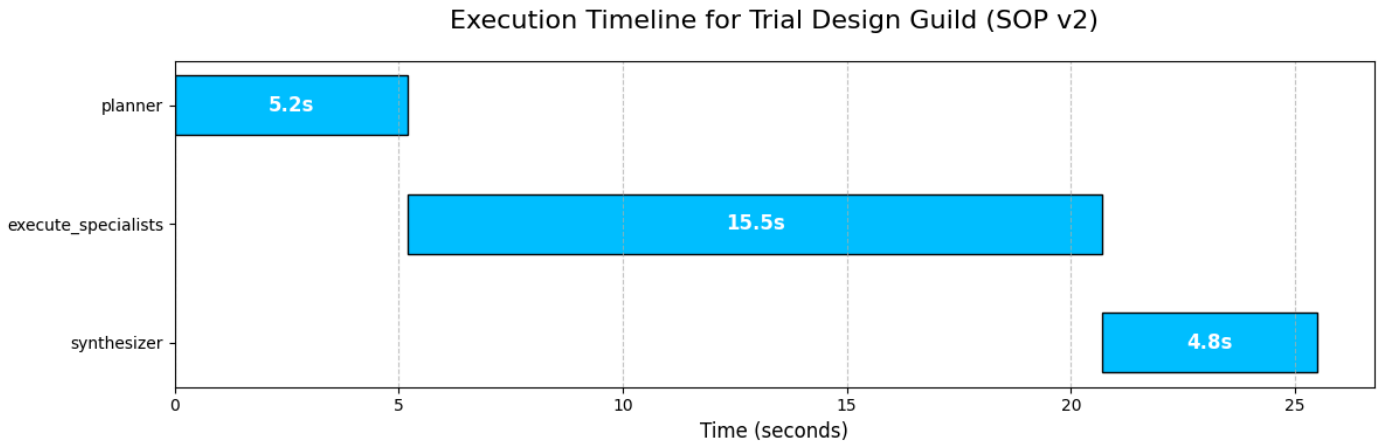
```
 matplotlib.pyplot  plt ():        fig, ax = plt.subplots(figsize=(, ))            labels
= [d[]  d  timing_data]                ax.barh(labels, [d[]  d  timing_data], left=[d[]  d
timing_data], color=)        ax.set_xlabel()    ax.set_title(title, fontsize=)
ax.grid(, which=, axis=, linestyle=, alpha=)            ax.invert_yaxis()    plt.show()
```

This `plot_gantt_chart` function is a standard Matplotlib plotting utility. It takes our list of timing data dictionaries and uses `ax.barh` to create horizontal bars. The key is the `left` parameter, which offsets each bar to its correct start time, and the width of the bar, which is set to its `duration`. This simple technique is all that's needed to transform our numerical data into an intuitive timeline.

Let's run it with the data we just captured.

```
plot_gantt_chart(timing_data_v2, )
```

This Gantt chart provides a powerful and clear visualization of our Guild's internal workflow. It's the "black box" made visible. We can instantly see:




1. The process is sequential, moving from `planner` to `execute_specialists` to `synthesizer`, exactly as we designed in our `LangGraph`.
2. The `execute_specialists` node is, by far, the longest-running part of the process. This is completely expected, as it involves multiple, independent sub-tasks (four different agent calls, some involving RAG and one involving a Text-to-SQL pipeline).
3. While our top-level graph is sequential, this visualization makes it clear that the the `execute_specialists` node is where parallelization happens. If we were to instrument the sub-tasks within that node, we would see the four specialist agents running concurrently.

This kind of timeline analysis is critical for performance optimization. It immediately tells us that if we want to make our Guild faster, our efforts should be focused almost exclusively on optimizing the `execute_specialists` node.

## Profiling the Output with a Radar Chart

The Gantt chart showed us the performance of the *process*. Now, let's visualize the performance of the outcome. Our evaluation system produces a 5D vector of scores. A simple table of numbers is hard to interpret. A **Radar Chart** (or Spider Plot) is a perfect tool for this, as it can map a multi-dimensional profile onto an intuitive, 2D shape.

We will write a function that takes our `EvaluationResult` objects and plots their 5D score vectors on a single radar chart. This will allow us to visually compare the performance profiles of different SOPs.

```
 pandas  pd ():                categories = [, , , , ]    num_vars = (categories)
angles = np.linspace(,  * np.pi, num_vars, endpoint=).tolist()        angles += angles[:]
fig, ax = plt.subplots(figsize=(, ), subplot_kw=(polar=))         i, result
(eval_results):                values = [res.score  res  result.().values()]        values
+= values[:]        ax.plot(angles, values, linewidth=, linestyle=, label=labels[i])
ax.fill(angles, values, alpha=)    ax.set_yticklabels([])    ax.set_xticks(angles[:-])
ax.set_xticklabels(categories, fontsize=)    ax.set_title(, size=, color=, y=)
plt.legend(loc=, bbox_to_anchor=(, ))    plt.show()
```

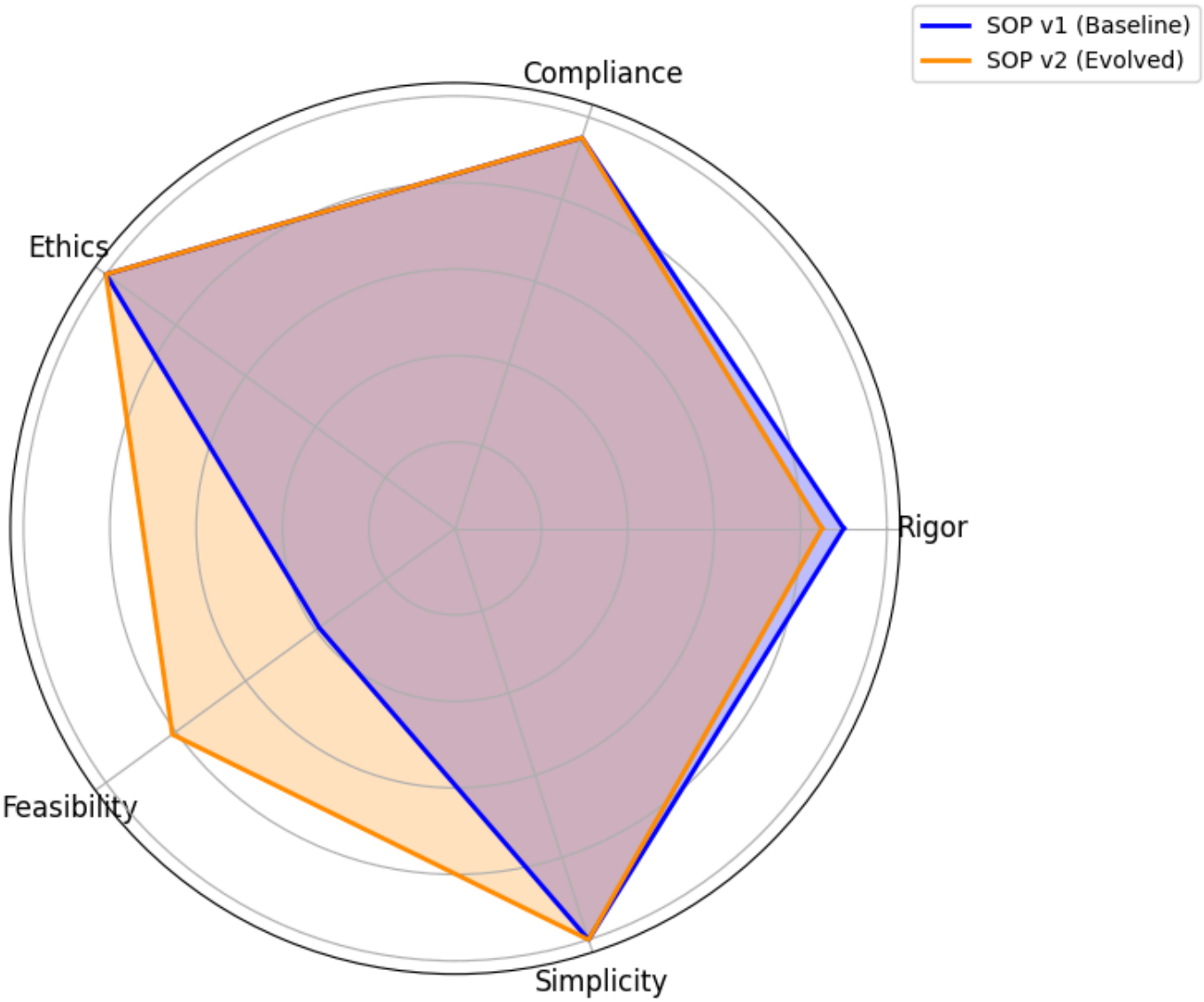
We ar again using uses Matplotlib `polar=True` projection to create the circular layout. The core logic involves calculating the angles for each of our five categories and then plotting each SOP's scores as a line that connects these angles.

The `ax.fill` command adds the semi-transparent color, making the "footprint" of each SOP's performance profile easy to see and compare.

Let’s run this function to compare our baseline `SOP v1` against our evolved, successful `SOP v2`.

```
evals_to_plot = [    gene_pool.pool[][],     gene_pool.pool[][]  ]labels_for_plot = [,
]plot_radar_chart(evals_to_plot, labels_for_plot)
```

It translates the abstract, 5D performance vectors into an immediate, intuitive comparison of strategic profiles.




- We can instantly see that both the baseline SOP (blue) and the evolved SOP (orange) are nearly perfect on “Compliance”, “Ethics,” and “Simplicity.” Their shapes both extend to the outer edge of these three axes. This tells us that our initial design was already strong in these areas.
- The key insight comes from the “Rigor” and “Feasibility” axes. We can see a clear trade-off. The blue shape () extends slightly further out on the “Rigor” axis, confirming its higher score. However, the orange shape () bulges out dramatically on the “Feasibility” axis, visually representing its massive improvement on that metric.

This chart is the final report for our AI Research Director. It proves that the evolution was not random; it was a targeted, intelligent optimization.

1. The system identified a specific weakness (Feasibility) and successfully evolved a new process (SOP v2) that dramatically improved it, while making a minimal, strategic sacrifice in another area (Rigor).
2. This visualization makes the complex, multi-objective trade-off clear, allowing a human expert to confidently choose SOP v2 as the superior overall strategy.

## Making it an Autonomous Strategy

We have successfully designed, built, and demonstrated a complete, self-improving agentic system. This architecture is not just a solution; it's a foundation. The principles we have established hierarchical agent design, dynamic SOPs, multi-dimensional evaluation, and automated evolution open up a vast number of future possibilities.

This is how I think we can take this research next:

1. We have completed one cycle, the next step is to run this for hundreds of generations to discover a richer and more diverse Pareto Frontier of optimal, battle-tested SOPs.
2. By training on the history of successful mutations, we could replace the large 70B Director LLM with a faster, cheaper, and specialized model to make the evolution process more efficient.
3. The Director could learn to add new specialists (like a "Biostatistician") or remove others based on the specific demands of a trial concept, evolving the team itself.
4. Connecting the `Patient Cohort Analyst` to a secure, real-time Electronic Health Record (EHR) system would ground its feasibility estimates in the most current patient data available.
5. `SOP Architect` Instead of just generating mutations, it could learn to use techniques like "crossover" to combine the best parts of two different successful SOPs, accelerating the discovery of novel strategies.
6. We could integrate a human clinical scientist's scores directly into the evaluation gauntlet, using their expert judgment as the ultimate reward signal to guide the system towards solutions that are not just technically optimal but also practically brilliant.

> You can follow me on Medium if you find this article useful